Министерство науки и высшего образования Российской Федерации
ФГБОУ ВО «Кубанский государственный технологический университет»
(ФГБОУ ВО «КубГТУ»)

Институт [наименование института]

Кафедра информационных систем и программирования

Направление подготовки 09.03.04 Программная инженерия

Профиль [наименование профиля]

# КУРСОВАЯ РАБОТА

по дисциплине «Системы искусственного интеллекта»

**на тему: «Разработка гибридной интеллектуальной системы планирования маршрутов доставки на основе генетического алгоритма и нечёткой логики»**

Выполнил(-а) студент(-ка) [курс] курса [группа] группы
[Фамилия Имя Отчество]

Допущен к защите _____

Руководитель (нормоконтролёр) работы
[должность] _____ [Фамилия И.О.]

Защищен _____ Оценка _____

Члены комиссии _____

_____

_____

(подпись, дата, расшифровка подписи)

Краснодар
2026 г.

ФГБОУ ВО «Кубанский государственный технологический университет»
(ФГБОУ ВО «КубГТУ»)

Институт [наименование института]
Кафедра информационных систем и программирования
Направление подготовки 09.03.04 Программная инженерия
Профиль [наименование профиля]

УТВЕРЖДАЮ
Зав. кафедрой _____
«___» _____ 20__ г.

**ЗАДАНИЕ**
на курсовую работу

Студенту(-ке) [Фамилия И.О.] [курс] курса группы [группа]
Тема работы: «Разработка гибридной интеллектуальной системы планирования маршрутов доставки на основе генетического алгоритма и нечёткой логики»
(утверждена указанием директора института № ___ от _____ 20__ г.)

План работы:
1. Анализ проблем предметной области: задача маршрутизации транспорта с ограничениями вместимости и временными окнами, критерии качества плана доставки.
2. Выбор и обоснование интеллектуальных методов: генетический алгоритм и нечёткий логический вывод Мамдани; гибридная схема их взаимодействия.
3. Разработка интеллектуальной системы (программная реализация на Python) и экспериментальный анализ её работы.

Объём работы:
а) пояснительная записка 58 с.;
б) иллюстративная часть _____ лист(-ов).

Рекомендуемая литература:
1. Джонс, М. Т. Программирование искусственного интеллекта в приложениях / М. Т. Джонс ; пер. с англ. А. И. Осипова. – М. : ДМК Пресс, 2011. – 312 с.
2. Рутковская, Д. Нейронные сети, генетические алгоритмы и нечеткие системы / Д. Рутковская, М. Пилиньский, Л. Рутковский ; пер. с польск. И. Д. Рудинского. – М. : Горячая линия – Телеком, 2006. – 452 с.
3. Гладков, Л. А. Генетические алгоритмы / Л. А. Гладков, В. В. Курейчик, В. М. Курейчик ; под ред. В. М. Курейчика. – 2-е изд., испр. и доп. – М. : Физматлит, 2006. – 320 с.

Срок выполнения работы: с «___» _____ по «___» _____ 20__ г.
Срок защиты: «___» _____ 20__ г.
Дата выдачи задания: «___» _____ 20__ г.
Дата сдачи работы на кафедру: «___» _____ 20__ г.

Руководитель работы _____ [Фамилия И.О.]
(должность, подпись)

Задание принял студент _____ [Фамилия И.О.]
(подпись)

# РЕФЕРАТ

Курсовая работа: 79 с., 14 рис., 13 табл., 24 источников, 3 приложения.

ИНТЕЛЛЕКТУАЛЬНАЯ СИСТЕМА, ГЕНЕТИЧЕСКИЙ АЛГОРИТМ, НЕЧЁТКАЯ ЛОГИКА, АЛГОРИТМ МАМДАНИ, ЛИНГВИСТИЧЕСКАЯ ПЕРЕМЕННАЯ, БАЗА ПРАВИЛ, ГИБРИДНАЯ СИСТЕМА, ЗАДАЧА МАРШРУТИЗАЦИИ ТРАНСПОРТА, ВРЕМЕННЫЕ ОКНА, ЛОГИСТИКА

Объект исследования – процесс планирования маршрутов доставки грузов парком транспортных средств с ограниченной грузоподъёмностью при наличии временных окон и приоритетов (срочности) заказов. Предмет исследования – гибридные методы искусственного интеллекта для поиска и оценки планов доставки.

Цель работы – разработка интеллектуальной системы, в которой генетический алгоритм ищет распределение заказов по машинам и порядок их объезда, а система нечёткого вывода Мамдани с базой из 26 экспертных правил оценивает качество каждого плана и тем самым задаёт функцию приспособленности.

Методы: генетический алгоритм с перестановочным кодированием, упорядоченным скрещиванием, тремя операторами мутации и элитизмом; нечёткая логика (лингвистические переменные, продукционная база правил с весовыми коэффициентами, алгоритм Мамдани с дефаззификацией по центру тяжести); вычислительный эксперимент.

Результат: программная система на языке Python с графическим и консольным интерфейсом. На задачах с 50 клиентами система сокращает суммарное опоздание по сравнению с жадным алгоритмом «ближайший сосед» на 95 %, а на малых задачах находит точный оптимум, подтверждённый полным перебором.

**СОДЕРЖАНИЕ**

ВВЕДЕНИЕ.....6
НОРМАТИВНЫЕ ССЫЛКИ.....9
ТЕРМИНЫ, ОПРЕДЕЛЕНИЯ И СОКРАЩЕНИЯ.....10
1 Анализ проблем предметной области.....12
1.1 Логистика «последней мили» и задача планирования доставки.....12
1.2 Задача маршрутизации транспорта и её разновидности.....13
1.3 Вычислительная сложность задачи.....14
1.4 Критерии качества плана доставки и их конфликт.....16
1.5 Лицо, принимающее решение, и необходимые ему зависимости.....17
1.6 Постановка задачи курсовой работы.....18
2 Выбор и применение интеллектуального метода.....20
2.1 Обзор подходов искусственного интеллекта.....20
2.2 Обоснование выбора генетического алгоритма и нечёткой логики.....21
2.3 Обоснование гибридной схемы.....22
2.4 Генетические алгоритмы.....23
2.5 Нечёткая логика и алгоритм нечёткого вывода Мамдани.....25
3 Разработка интеллектуальной системы.....29
3.1 Архитектура системы.....29
3.2 Модель задачи и генератор исходных данных.....31
3.3 Нечёткая модель оценки качества плана.....31
3.3.1 Лингвистические переменные.....31
3.3.2 База нечётких правил.....34
3.3.3 Реализация механизма вывода.....36
3.4 Генетический алгоритм.....38
3.4.1 Кодирование решения.....38
3.4.2 Функция приспособленности.....39
3.4.3 Генетические операторы.....39
3.4.4 Начальная популяция, элитизм и останов.....40
3.5 Пользовательский интерфейс.....41

4 Экспериментальный анализ.....43
4.1 Методика экспериментов.....43
4.2 Сравнение с базовыми методами.....43
4.3 Сравнение с полным перебором.....45
4.4 Сходимость генетического алгоритма.....46
4.5 Влияние параметров генетического алгоритма.....47
4.6 Пример работы системы и протокол нечёткого вывода.....49
4.7 Работа программы.....52
ЗАКЛЮЧЕНИЕ.....54
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....56
Приложение А Листинг основных модулей программы.....59
Приложение Б Пример входного файла задачи.....77
Приложение В Пример протокола работы программы.....78

# ВВЕДЕНИЕ

Доставка товаров конечному получателю – так называемая «последняя миля» – остаётся самым дорогим звеном логистической цепочки. Рост электронной торговли и сервисов доставки продуктов, лекарств и запасных частей привёл к тому, что ежедневно диспетчер службы доставки должен распределить десятки заказов между несколькими машинами и определить порядок их объезда. При этом план должен одновременно учитывать грузоподъёмность транспорта, интервалы времени, в которые клиенты готовы принять груз, приоритет срочных заказов и равномерность загрузки водителей. Даже для небольшого числа клиентов количество вариантов плана исчисляется миллиардами, а критерии качества противоречат друг другу, поэтому составление плана «вручную» или простыми правилами приводит к перерасходу пробега и опозданиям.

Математической моделью этой задачи является задача маршрутизации транспортных средств (Vehicle Routing Problem, VRP) с ограничениями вместимости и временными окнами. Она относится к классу NP-трудных задач [1], и точные методы применимы к ней лишь при малой размерности. Поэтому на практике используются эвристические и метаэвристические методы, среди которых важное место занимают методы искусственного интеллекта, в частности генетические алгоритмы [2, 3]. Однако генетический алгоритм требует функции приспособленности – числовой меры качества решения, а качество плана доставки является многокритериальным и во многом субъективным понятием: логист оценивает план словами «пробег великоват, но опозданий нет», «срочные заказы доставлены слишком поздно». Формализовать такие оценки позволяет аппарат нечёткой логики [4, 5].

**Актуальность** работы определяется практической потребностью служб доставки в автоматизированном построении планов, учитывающих несколько противоречивых критериев, и методической потребностью в

демонстрации гибридных интеллектуальных систем, сочетающих поисковые и экспертные методы искусственного интеллекта.

**Объект исследования** – процесс планирования маршрутов доставки грузов парком транспортных средств.

**Предмет исследования** – гибридные методы искусственного интеллекта (генетический алгоритм и нечёткий логический вывод) для поиска и оценки планов доставки.

**Цель работы** – разработать гибридную интеллектуальную систему, в которой генетический алгоритм ищет решение задачи маршрутизации (распределение заказов по машинам и порядок объезда), а нечёткий вывод по алгоритму Мамдани вычисляет оценку качества маршрута, служащую функцией приспособленности генетического алгоритма.

Для достижения цели поставлены следующие **задачи**:

– проанализировать предметную область, сформулировать задачу маршрутизации с временными окнами, критерии качества плана и зависимости между ними;

– выполнить обзор методов искусственного интеллекта и обосновать выбор генетического алгоритма и нечёткой логики, а также гибридной схемы их взаимодействия;

– разработать нечёткую модель оценки качества плана: лингвистические переменные, функции принадлежности и базу продукционных правил с весовыми коэффициентами;

– разработать генетический алгоритм: способ кодирования решения, генетические операторы, критерии останова;

– реализовать программную систему с графическим и консольным интерфейсом;

– провести экспериментальный анализ: сравнить систему с базовыми методами и с точным решением, исследовать влияние параметров генетического алгоритма.

**Методы исследования**: теория нечётких множеств и нечёткий логический вывод, эволюционные вычисления, имитационное моделирование движения транспорта, вычислительный эксперимент с последующей статистической обработкой результатов.

**Практическая значимость** работы состоит в том, что разработанная система может использоваться диспетчером службы доставки как инструмент поддержки принятия решений: она строит план за несколько секунд и объясняет полученную оценку в терминах, понятных логисту, – через сработавшие правила базы знаний. Система написана на языке Python [6] с использованием библиотеки NumPy [7]; генетический алгоритм и механизм нечёткого вывода реализованы без сторонних библиотек искусственного интеллекта, что делает каждый этап вычислений прозрачным.

Пояснительная записка состоит из четырёх разделов. В первом разделе анализируется предметная область, во втором обосновывается выбор методов и излагается их теория, в третьем описывается разработанная система, в четвёртом приводятся результаты экспериментов.

## НОРМАТИВНЫЕ ССЫЛКИ

В настоящей курсовой работе использованы ссылки на следующие нормативные документы:

– ГОСТ Р 7.0.5-2008 СИБИД. Библиографическая ссылка. Общие требования и правила составления;

– ГОСТ 2.105-2019 ЕСКД. Общие требования к текстовым документам;

– ГОСТ 7.9-95 СИБИД. Реферат и аннотация. Общие требования;

– ГОСТ 7.82-2001 СИБИД. Библиографическая запись. Библиографическое описание электронных ресурсов. Общие требования и правила составления;

– ГОСТ 19.701-90 ЕСПД. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения.

## ТЕРМИНЫ, ОПРЕДЕЛЕНИЯ И СОКРАЩЕНИЯ

В настоящей пояснительной записке применяют следующие термины с соответствующими определениями и сокращения.

| | |
|---|---|
| Аккумуляция | – Этап нечёткого вывода: объединение (операция max) активизированных нечётких множеств всех правил в одно множество выходной переменной |
| Активизация | – Этап нечёткого вывода: вычисление степени истинности заключения правила с учётом весового коэффициента и «срез» функции принадлежности терма заключения |
| ВО | – Временное окно – интервал времени [e; l], в течение которого клиент может принять груз |
| ГА | – Генетический алгоритм – эвристический метод поиска, имитирующий естественный отбор: популяция решений улучшается операторами отбора, скрещивания и мутации |
| Ген, хромосома, особь | – Элемент кодированного решения; закодированное решение; решение в составе популяции ГА |
| Дефаззификация | – Переход от нечёткого множества к чёткому числу; в работе – метод центра тяжести |
| Лингвистическая переменная | – Переменная, значениями которой являются слова (термы), каждое из которых задано нечётким множеством |
| ЛПР | – Лицо, принимающее решение (диспетчер, логист службы доставки) |
| Популяция | – Множество особей, одновременно обрабатываемых ГА на одном шаге (поколении) |
| Приспособленность | – Числовая оценка качества особи, по которой ведётся отбор в ГА |
| Терм | – Значение лингвистической переменной («короткая», |

| | |
|---|---|
| | «средняя», «длинная») |
| Фаззификация | – Вычисление степеней принадлежности чётких входных значений термам лингвистических переменных |
| Функция принадлежности | – Функция $\mu(x) \in [0; 1]$, задающая степень принадлежности элемента x нечёткому множеству |
| Элитизм | – Перенос лучших особей в следующее поколение без изменений |
| CVRP | – Capacitated Vehicle Routing Problem – задача маршрутизации с ограничением вместимости |
| OX | – Упорядоченное скрещивание (Order Crossover) – оператор скрещивания для перестановок |
| VRP | – Vehicle Routing Problem – задача маршрутизации транспортных средств |
| VRPTW | – Vehicle Routing Problem with Time Windows – задача маршрутизации с временными окнами |

# 1 Анализ проблем предметной области

## 1.1 Логистика «последней мили» и задача планирования доставки

Логистикой «последней мили» называют завершающий этап доставки – перемещение груза от склада или распределительного центра до конечного получателя. Этот этап характеризуется большим числом мелких отправок, рассредоточенных по территории города, жёсткими требованиями клиентов ко времени доставки и высокой долей затрат на топливо и оплату труда водителей. Типичная служба доставки (интернет-магазин, аптечная сеть, поставщик запасных частей) располагает одним складом и небольшим парком однотипных машин, которые утром загружаются на складе, развозят заказы и к концу смены возвращаются.

Каждый рабочий день диспетчер получает список заказов. О каждом заказе известны адрес клиента, объём груза, интервал времени, в который клиент готов принять груз (временное окно), и признак срочности. Диспетчеру требуется решить две связанные задачи: распределить заказы по машинам так, чтобы ни одна машина не была перегружена, и для каждой машины определить порядок объезда клиентов. Вместе эти решения образуют *план доставки*. Качество плана определяет себестоимость доставки (пробег, время работы водителей) и уровень сервиса (доставка в обещанное время, приоритет срочных заказов).

Основные понятия предметной области, используемые далее, сведены в таблицу 1.

Таблица 1 – Основные понятия предметной области

| Понятие | Содержание и обозначение в модели |
|---|---|
| Склад (депо) | Точка, в которой машины начинают и заканчивают смену; вершина 0 графа |
| Клиент | Точка доставки $i = 1, \ldots, N$ с координатами, объёмом заказа $q_i$, временным окном $[e_i; l_i]$, срочностью $u_i \in \{1, 2, 3\}$ и временем разгрузки |
| Транспортное средство | Машина вместимостью Q; парк из K одинаковых машин со скоростью v |
| Маршрут | Последовательность клиентов, объезжаемых одной машиной: |

| Понятие | Содержание и обозначение в модели |
|---|---|
| | склад → клиенты → склад |
| План доставки (решение) | Совокупность K маршрутов, в которой каждый клиент посещён ровно один раз |
| Временное окно | Интервал, в который клиент может принять груз; ранний приезд вызывает ожидание, поздний – опоздание |
| Срочный заказ | Заказ, который желательно доставить в начале смены (лекарства, скоропортящиеся товары) |
| Смена | Интервал работы [0; H], H = 480 мин |

### 1.2 Задача маршрутизации транспорта и её разновидности

Задача планирования доставки является классической задачей исследования операций – задачей маршрутизации транспортных средств (VRP), впервые сформулированной Дж. Данцигом и Дж. Рамсером в 1959 году как задача о диспетчеризации грузовиков [8]. Уже через несколько лет Г. Кларк и Дж. Райт предложили для неё эвристику сбережений, ставшую стандартным инструментом логистов [9]. С тех пор сформировалось целое семейство постановок [10]:

– CVRP – задача с ограничением вместимости: суммарный объём заказов на маршруте не превышает вместимости машины;

– VRPTW – задача с временными окнами: каждый клиент должен быть обслужен в заданный интервал времени; систематическое исследование эвристик для этого варианта выполнено М. Соломоном [11];

– многокритериальные варианты, в которых наряду с пробегом учитываются опоздания, приоритеты заказов и равномерность загрузки.

Пусть $x_{ijk} = 1$, если машина k проезжает из точки i непосредственно в точку j, и $d_{ij}$ – расстояние между точками. Классическая постановка CVRP требует минимизировать суммарный пробег

$$min \sum_{k=1}^{K} \sum_{i,} \sum_{j} d_{ij} \cdot x_{ijk} \qquad (1)$$

при ограничении вместимости для каждого маршрута $R_k$

$$\sum_{i \in R_k} q_i \le Q, k = 1, \ldots, K \quad (2)$$

и условиях, что каждый клиент посещается ровно один раз, а каждый маршрут начинается и заканчивается на складе. В варианте с временными окнами для каждого клиента вычисляется момент начала обслуживания $s_i$: если машина прибыла в момент $t_i$ раньше начала окна $e_i$, она ожидает; если позже конца окна $l_i$ – возникает опоздание:

$$s_i = max(e_i, t_i), late_i = max(0, s_i - l_i) \quad (3)$$

В «жёсткой» постановке VRPTW опоздания запрещены. В реальной практике окна часто являются «мягкими»: опоздание допускается, но ухудшает качество обслуживания. В данной работе ограничение вместимости считается жёстким (машину физически нельзя перегрузить), а временные окна – мягкими.

### 1.3 Вычислительная сложность задачи

Задача VRP обобщает задачу коммивояжёра (при K = 1 и неограниченной вместимости) и, как доказано Я. Ленстрой и А. Ринной Каном, является NP-трудной [1]: для неё не известно алгоритма, время работы которого росло бы полиномиально с ростом числа клиентов. Оценим размер пространства решений. Если записать план как перестановку N клиентов, разделённую K − 1 одинаковыми разделителями на K маршрутов, то число различных записей равно

$$S(N, K) = \frac{(N + K - 1)!}{(K - 1)!} \quad (4)$$

Рост этой величины показан в таблице 2 и на рисунке 1. Уже при N = 20 полный перебор со скоростью миллиард вариантов в секунду занял бы тысячи лет, а при N = 50 время перебора превышает возраст Вселенной на десятки порядков.

Таблица 2 – Размер пространства решений задачи маршрутизации

| N | K | Число хромосом (N+K−1)!/(K−1)! | Время перебора при 10^9 решений/с |
|---|---|---|---|
| 5 | 2 | 7,20·10^2 | 0,000 с |
| 8 | 2 | 3,63·10^5 | 0,000 с |
| 10 | 2 | 3,99·10^7 | 0,040 с |
| 20 | 3 | 5,62·10^20 | ≈ 10^4 лет |
| 30 | 4 | 1,45·10^36 | ≈ 10^19 лет |
| 50 | 7 | 9,87·10^71 | ≈ 10^55 лет |

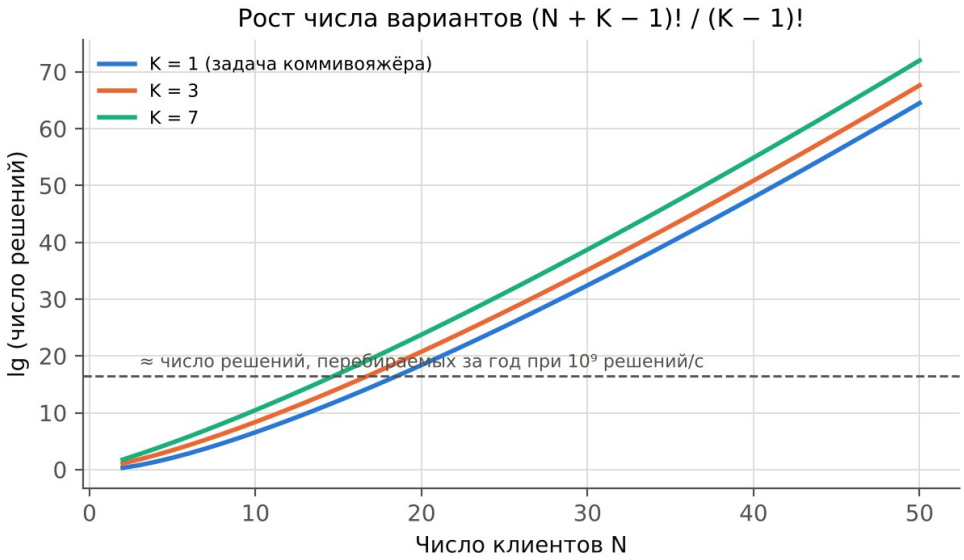



Рисунок 1 – Рост числа допустимых записей плана с увеличением числа клиентов

Точные методы (ветвей и границ, динамическое программирование, целочисленное программирование) применимы к задачам с несколькими десятками клиентов лишь ценой больших вычислительных затрат [10]. Для повседневного планирования, когда план нужен за секунды, применяются эвристики (ближайший сосед, сбережения Кларка – Райта) и метаэвристики – генетические алгоритмы, имитация отжига, поиск с запретами [12]. Эвристики работают быстро, но учитывают лишь один критерий (обычно пробег) и не гарантируют качества; метаэвристики позволяют работать с произвольной целевой функцией, в том числе многокритериальной.

### 1.4 Критерии качества плана доставки и их конфликт

На практике качество плана не сводится к пробегу. Анализ требований служб доставки позволяет выделить пять критериев, приведённых в таблице 3.

Таблица 3 – Критерии качества плана доставки

| **Критерий** | **Показатель в модели** | **Желаемое значение** | **Кому важен** |
|---|---|---|---|
| Протяжённость маршрутов | Суммарный пробег D, отнесённый к пробегу эталонного решения | Меньше | Бухгалтерия (топливо, износ) |
| Соблюдение временных окон | Средняя по клиентам относительная величина опоздания | Нулевая | Клиенты, отдел продаж |
| Обслуживание срочных заказов | Среднее время доставки срочных заказов относительно смены | Раньше | Клиенты со срочными заказами |
| Загрузка транспорта | Максимальная загрузка машины в долях вместимости | Не более 1 | Водители, техника безопасности |
| Сбалансированность | Коэффициент вариации длительностей маршрутов | Близок к 0 | Водители, отдел кадров |

Критерии конфликтуют между собой (рисунок 2). Кратчайший по пробегу объезд соседних клиентов игнорирует их временные окна: машина приезжает к клиенту, окно которого уже закрыто, или ожидает открытия окна, теряя время. Если доставлять срочные заказы первыми, обычные клиенты обслуживаются позже, и часть их окон нарушается. Выравнивание длительности маршрутов между водителями заставляет передавать «чужих» клиентов другой машине, что удлиняет маршруты. Наконец, ограничение вместимости вообще не допускает компромисса: перегруженный план неосуществим.

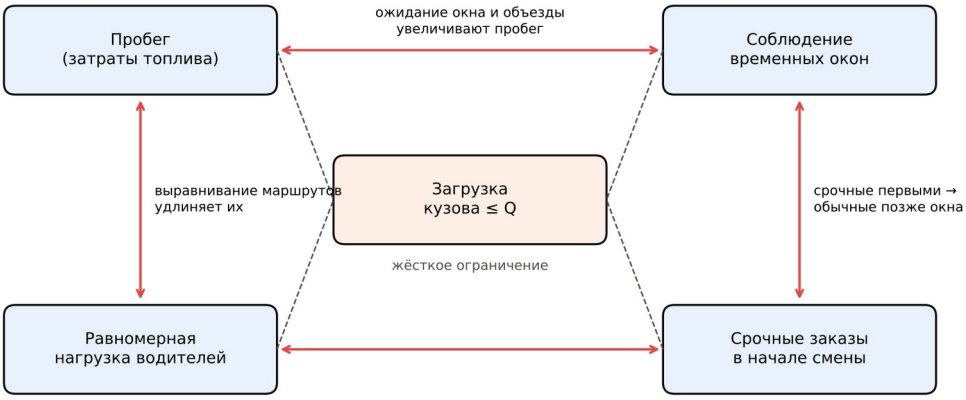



Рисунок 2 – Конфликт критериев качества плана доставки

Следовательно, сравнение двух планов требует свёртки нескольких разнородных показателей в одну оценку. Простейшая свёртка – взвешенная сумма – предполагает, что критерии взаимозаменяемы с постоянным «курсом обмена»: уменьшение опозданий на единицу всегда компенсирует фиксированное увеличение пробега. Однако логист рассуждает иначе: «если опозданий нет, лишние 5 % пробега допустимы; если опоздания массовые, никакая экономия пробега план не спасает». Такие нелинейные, качественные зависимости естественно описываются правилами вида «ЕСЛИ … ТО …» над нечёткими понятиями [5].

### 1.5 Лицо, принимающее решение, и необходимые ему зависимости

Лицом, принимающим решение, в рассматриваемой системе является диспетчер (логист) службы доставки. Он утверждает план на смену и отвечает перед руководством за затраты, а перед клиентами – за своевременность доставки. Для обоснованного решения ЛПР должен знать зависимости, приведённые в таблице 4; именно они затем формализуются в базе нечётких правил (подраздел 3.3).

Таблица 4 – Зависимости, которые должен учитывать ЛПР

| Ситуация | Последствие | Вывод для оценки плана |
|---|---|---|
| Пробег близок к минимальному, опозданий нет | Минимальные затраты при полном сервисе | Качество очень высокое |

| Ситуация | Последствие | Вывод для оценки плана |
|---|---|---|
| Пробег короткий, опоздания массовые | Экономия топлива ценой потери клиентов | Качество не выше среднего |
| Пробег большой, опозданий нет | Сервис обеспечен за счёт перерасхода | Качество среднее |
| Срочные заказы доставлены в конце смены | Риск отказа клиента и штрафов | Качество снижается |
| Хотя бы одна машина перегружена | План физически неосуществим | Качество очень низкое, план отвергается |
| Один водитель работает вдвое дольше другого | Сверхурочные, опоздания у перегруженного водителя | Качество снижается |
| Машины недогружены, но пробег большой | Парк используется неэффективно | Качество низкое |

Эти зависимости носят качественный характер: понятия «пробег близок к минимальному», «опоздания массовые», «доставлены в конце смены» не имеют чётких границ. Перевод их в числовые пороги (например, «массовые – более 20 % клиентов») неизбежно произволен и создаёт скачки оценки при переходе через порог. Это обстоятельство определяет выбор аппарата нечёткой логики для оценки планов (раздел 2).

### 1.6 Постановка задачи курсовой работы

С учётом проведённого анализа задача, решаемая разрабатываемой системой, формулируется следующим образом. Задан склад и N клиентов на плоскости; для каждого клиента известны координаты, объём заказа, срочность и временное окно. Задан парк из K одинаковых машин вместимостью Q, движущихся с постоянной скоростью; все машины начинают и заканчивают смену на складе. Требуется построить план доставки, который:

– не нарушает ограничения вместимости ни для одной машины (жёсткое ограничение);

– имеет максимальную оценку качества, учитывающую протяжённость маршрутов, соблюдение временных окон, обслуживание срочных заказов, загрузку транспорта и сбалансированность нагрузки водителей.

Оценка качества должна формироваться по правилам, понятным ЛПР, а поиск плана – выполняться за время, приемлемое для ежедневного планирования (секунды). Исходные данные задач генерируются программно с фиксированным начальным значением генератора случайных чисел и сохраняются в формате JSON, что обеспечивает воспроизводимость экспериментов.

# 2 Выбор и применение интеллектуального метода

## 2.1 Обзор подходов искусственного интеллекта

Современные интеллектуальные системы, как правило, являются гибридными: они сочетают два и более подхода искусственного интеллекта, каждый из которых решает свою часть задачи [13, 14]. Методические указания перечисляют следующие подходы: классические экспертные системы с продукционной моделью знаний, семантические и фреймовые модели, нейросетевые системы распознавания образов и ассоциативной памяти, нейроконтроллеры, нечёткую логику и алгоритмы нечёткого вывода, методы интеллектуального анализа данных (data mining), игровые алгоритмы и эвристики, естественно-языковые системы, агентные и мультиагентные системы, генетические и другие эволюционные алгоритмы [15, 16].

Задача, сформулированная в подразделе 1.6, распадается на две подзадачи разной природы:

– **поиск** плана в огромном дискретном пространстве перестановок (подраздел 1.3);

– **оценка** плана по нескольким противоречивым критериям на основе качественных знаний ЛПР (подразделы 1.4–1.5).

Применимость перечисленных подходов к каждой из подзадач сопоставлена в таблице 5.

Таблица 5 – Применимость подходов искусственного интеллекта к задаче

| Подход | Поиск плана | Оценка плана | Вывод |
|---|---|---|---|
| Продукционная экспертная система | Не ищет решения, только оценивает | Правила ЛПР выражаются естественно, но чёткие пороги дают скачки оценки | Частично |
| Семантические и фреймовые модели | Не предназначены для оптимизации | Описывают понятия, но не дают числовой оценки | Не подходит |
| Нейронные сети | Требуют обучающей выборки оптимальных планов, которой нет | Можно обучить оценку, но нужна размеченная выборка; | Не подходит |

| Подход | Поиск плана | Оценка плана | Вывод |
|---|---|---|---|
| | | «чёрный ящик» | |
| Нечёткая логика (Мамдани) | Не ищет решения | Формализует качественные правила ЛПР, оценка плавная и объяснимая | Подходит для оценки |
| Data mining (кластеризация) | Кластеризация клиентов – лишь предварительный шаг | Не оценивает план | Вспомогательно |
| Игровые алгоритмы, эвристики | Быстры (ближайший сосед), но учитывают один критерий | Не оценивают | Базовый уровень |
| Агентные системы | Возможны (агент-машина), но сложны в реализации и отладке | Нужна общая функция оценки | Избыточно |
| Генетические алгоритмы | Эффективный поиск в пространстве перестановок с любой функцией качества | Требуют внешней функции приспособленности | Подходит для поиска |

Из таблицы видно, что ни один подход не решает задачу целиком. Генетический алгоритм хорошо ищет, но «не знает», что такое хороший план; нечёткая система вывода хорошо оценивает, но не умеет искать. Нейросетевые методы и методы интеллектуального анализа данных [17] требуют больших выборок данных, которых в постановке задачи нет, а агентные подходы, широко применяемые в робототехнике [18], для статической задачи планирования избыточны.

## 2.2 Обоснование выбора генетического алгоритма и нечёткой логики

**Почему генетический алгоритм, а не точный метод.** Как показано в подразделе 1.3, полный перебор невозможен уже при двадцати клиентах, а точные методы ветвей и границ требуют линейной (или специальной) целевой функции и большого времени. Оценка же плана, которую мы хотим оптимизировать, нелинейна и вычисляется алгоритмически (через моделирование движения машин и нечёткий вывод), поэтому методы математического программирования к ней неприменимы. Генетический

алгоритм не накладывает на целевую функцию никаких требований: ему достаточно уметь вычислить её значение для любого решения [2]. Он работает с популяцией решений, что снижает риск застревания в локальном оптимуме, и естественно представляет план перестановкой. Эффективность эволюционных алгоритмов для задач маршрутизации подтверждена многочисленными исследованиями [12, 3]. Ценой является отсутствие гарантии оптимальности: ГА находит хорошее, но, вообще говоря, не обязательно оптимальное решение (это проверяется экспериментально в подразделе 4.3).

**Почему нечёткая логика, а не взвешенная сумма.** Альтернативой нечёткой оценке является линейная свёртка показателей с весами. Её недостатки:

– взвешенная сумма предполагает постоянную взаимозаменяемость критериев, тогда как зависимости ЛПР нелинейны (таблица 4): опоздания при коротком пробеге и при длинном пробеге оцениваются по-разному;

– веса трудно назначить и невозможно объяснить: эксперт не может сказать, «сколько километров стоит одно опоздание»;

– результат свёртки не интерпретируется словами.

Нечёткая система вывода, напротив, хранит знания в виде правил «ЕСЛИ протяжённость короткая И соблюдение окон хорошее ТО качество очень высокое», которые эксперт формулирует и проверяет непосредственно [5, 19]. Плавные функции принадлежности устраняют скачки оценки на границах понятий, а протокол вывода показывает, какие правила и с какой степенью истинности определили оценку. Нечёткая логика позволяет также естественно выражать приоритеты через весовые коэффициенты правил F (подраздел 2.5).

### 2.3 Обоснование гибридной схемы

Известны различные способы объединения генетических алгоритмов и нечётких систем [20, 21]. В схеме «ГА для нечёткой системы» генетический

алгоритм настраивает параметры функций принадлежности или базу правил по обучающей выборке. В схеме «нечёткая система для ГА» нечёткий вывод используется внутри генетического алгоритма – для адаптивного управления его параметрами или для вычисления приспособленности.

В данной работе выбрана вторая схема в варианте **нечёткой функции приспособленности**: генетический алгоритм ищет план доставки, а система нечёткого вывода Мамдани вычисляет оценку качества плана $Q \in [0; 1]$, которая и является приспособленностью особи. Методы не независимы: на каждом поколении для каждой новой особи ГА декодирует хромосому в набор маршрутов, моделирует их выполнение, вычисляет пять показателей качества и вызывает нечёткий вывод. Таким образом, нечёткая система задаёт «ландшафт» поиска, а ГА движется по нему к лучшим планам. Обучающая выборка при этом не нужна: знания вносятся экспертом в виде правил, а оптимизация выполняется непосредственно на решаемой задаче.

Жёсткое ограничение вместимости дополнительно учитывается штрафом вне нечёткого блока (подраздел 3.4). Это сделано потому, что нечёткая оценка по своей природе «мягкая»: перегруженный план с прекрасными остальными показателями мог бы получить оценку выше, чем допустимый план со средними показателями, и ГА начал бы сходиться к неосуществимым решениям.

### 2.4 Генетические алгоритмы

Генетический алгоритм предложен Дж. Холландом [22] и популяризирован Д. Голдбергом [2]. Он имитирует естественный отбор: популяция кандидатных решений (особей) эволюционирует от поколения к поколению, причём более приспособленные особи чаще становятся родителями и передают свои признаки потомкам. Основные элементы ГА [3, 20]:

– **кодирование** – способ представления решения в виде хромосомы (строки генов); для задач маршрутизации естественно перестановочное кодирование, при котором хромосома – перестановка номеров клиентов;

– **функция приспособленности** f – числовая оценка качества особи, которую алгоритм максимизирует;

– **отбор** – выбор родителей с предпочтением более приспособленных особей;

– **скрещивание** (кроссинговер) – получение потомков, наследующих признаки двух родителей;

– **мутация** – случайное малое изменение особи, поддерживающее разнообразие популяции;

– **элитизм** – перенос лучших особей в следующее поколение без изменений, гарантирующий, что лучшее найденное решение не будет потеряно;

– **критерии останова** – предельное число поколений, отсутствие улучшения в течение заданного числа поколений (стагнация), лимит времени.

Наиболее распространены три способа отбора. При **пропорциональном (рулеточном)** отборе вероятность выбора i-й особи пропорциональна её приспособленности:

$$p_i = \frac{f_i}{\sum_{j=1}^{P} f_j} \tag{5}$$

где P – размер популяции. Недостаток рулетки – зависимость от масштаба значений f: если приспособленности близки, отбор почти случаен, а если одна особь много лучше остальных, она быстро вытесняет их. При **ранговом** отборе вероятность пропорциональна номеру особи в упорядоченной популяции, а не значению f. При **турнирном** отборе из популяции случайно выбираются k особей и родителем становится лучшая из

них; давление отбора регулируется размером турнира k и не зависит от масштаба f [2].

Обычные операторы скрещивания для двоичных строк (одноточечный, двухточечный) неприменимы к перестановкам: потомок может содержать один клиент дважды и не содержать другого. Для перестановок используются специальные операторы, среди которых наиболее известен **упорядоченный кроссовер** (OX) Л. Дэвиса [23]: отрезок хромосомы копируется из первого родителя, а оставшиеся гены добавляются в том порядке, в котором они встречаются во втором родителе. Так потомок наследует от одного родителя фрагмент маршрута, а от другого – относительный порядок остальных клиентов. Мутации для перестановок – обмен двух генов, инверсия отрезка (аналог известного улучшения маршрута 2-opt) и перенос гена в другую позицию.

Теоретическим обоснованием работы ГА служит теорема о схемах Холланда: короткие, малого порядка и высокоприспособленные фрагменты решений («строительные блоки») получают в следующих поколениях экспоненциально возрастающее число копий [22, 2]. Вместе с тем ГА – стохастический эвристический метод: он **не гарантирует нахождения глобального оптимума** за конечное время. Типичная проблема – преждевременная сходимость, когда популяция теряет разнообразие и застревает вблизи локального оптимума; ей противодействуют мутация, умеренное давление отбора и достаточный размер популяции.

### 2.5 Нечёткая логика и алгоритм нечёткого вывода Мамдани

Теория нечётких множеств предложена Л. Заде в 1965 году [4] как обобщение классической теории множеств для описания понятий с размытыми границами. Нечёткое множество A на универсуме X задаётся функцией принадлежности $\mu_A(x)$: $X \to [0; 1]$, значение которой – степень, с которой элемент x принадлежит множеству. Например, пробег,

превышающий эталонный на 5 %, можно считать «коротким» со степенью 0,5 и «средним» со степенью 0,5.

Чаще всего используются кусочно-линейные функции принадлежности [19]. Треугольная функция с параметрами a ≤ b ≤ c:

$$\mu(x)=max\left(0,min\left(\frac{x-a}{b-a},\frac{c-x}{c-b}\right)\right) \quad (6)$$

Трапециевидная функция с параметрами a ≤ b ≤ c ≤ d имеет плоскую вершину на отрезке [b; c]:

$$\mu(x)=max\left(0,min\left(\frac{x-a}{b-a},1,\frac{d-x}{d-c}\right)\right) \quad (7)$$

При a = b (или c = d) получаются «плечевые» функции, равные единице на краю универсума.

***Лингвистическая переменная*** – это переменная, значениями которой являются слова или словосочетания естественного языка (термы), а каждый терм задан нечётким множеством на общем универсуме [5]. Например, переменная «Протяжённость» имеет термы «короткая», «средняя», «длинная». Над нечёткими множествами определены операции: пересечение (логическое «И») обычно реализуется как минимум функций принадлежности, объединение («ИЛИ») – как максимум. Операция min выбрана Л. Заде как самая «осторожная» t-норма: степень истинности составного условия определяется самым слабым из подусловий («цепь не прочнее самого слабого звена»), и она не уменьшается искусственно с ростом числа подусловий, как это происходит при умножении.

Знания эксперта записываются в виде нечётких продукционных правил «ЕСЛИ условие ТО заключение» с весовыми коэффициентами F ∈ (0; 1], выражающими степень уверенности эксперта в правиле. Алгоритм нечёткого вывода Мамдани [24] состоит из шести последовательных этапов, каждый из которых получает на вход результат предыдущего [5, 21]:

1) **Формирование базы правил.** Задаются входные и выходные лингвистические переменные и n правил вида $RULE_j$: ЕСЛИ «условие$_1$» И … И «условие$_m$» ТО «заключение» ($F_j$).

2) **Фаззификация входных переменных.** Для массива чётких входных значений A = {$a_1$, …, $a_m$} вычисляются степени истинности всех подусловий базы правил:

$$b_i = \mu_i(a_i), i = 1, \ldots, k \tag{8}$$

где k – общее число подусловий.

3) **Агрегирование подусловий.** Для каждого правила определяется степень истинности его условия как минимум степеней истинности подусловий ($I_j$ – множество подусловий j-го правила):

$$c_j = \min_{i \in I_j}\{b_i\}, j = 1, \ldots, n \tag{9}$$

4) **Активизация подзаключений.** Степень истинности заключения умножается на весовой коэффициент правила, после чего функция принадлежности терма из заключения «срезается» на этом уровне (min-активизация):

$$d_j = c_j \cdot F_j, \mu'_j(x) = \min\{d_j, \mu_j(x)\} \tag{10}$$

5) **Аккумуляция заключений.** Активизированные множества всех правил объединяются операцией max в одно нечёткое множество E выходной переменной:

$$\mu_E(x) = \max_{j=1,\ldots,n}\{\mu'_j(x)\} \tag{11}$$

6) **Дефаззификация.** Нечёткое множество E преобразуется в чёткое число методом центра тяжести, где Min и Max – границы универсума выходной переменной:

$$y = \frac{\int_{Min}^{Max} x \cdot \mu_E(x)\,dx}{\int_{Min}^{Max} \mu_E(x)\,dx} \tag{12}$$

Метод центра тяжести учитывает вклад всех сработавших правил пропорционально «площади» их активизированных множеств, поэтому результат меняется плавно при плавном изменении входов. Это свойство особенно важно для использования нечёткой оценки в качестве функции приспособленности: ГА получает информацию не только о том, что одно решение лучше другого, но и о том, насколько.

К достоинствам нечётких систем относятся работа в условиях неопределённости, использование качественных экспертных знаний и интерпретируемость результата; к недостаткам – отсутствие формальной методики построения базы правил и зависимость результата от выбора функций принадлежности [5]. Последний недостаток в работе компенсируется экспериментальной проверкой поведения нечёткой оценки (раздел 4).

# 3 Разработка интеллектуальной системы

## 3.1 Архитектура системы

Система реализована на языке Python 3 [6] в виде набора модулей, каждый из которых отвечает за одну часть задачи (рисунок 3). Генетический алгоритм (пакет ga) и механизм нечёткого вывода (пакет fuzzy) написаны без использования сторонних библиотек искусственного интеллекта; библиотека NumPy [7] применяется только для векторных вычислений над сеткой выходной переменной и матрицей расстояний.

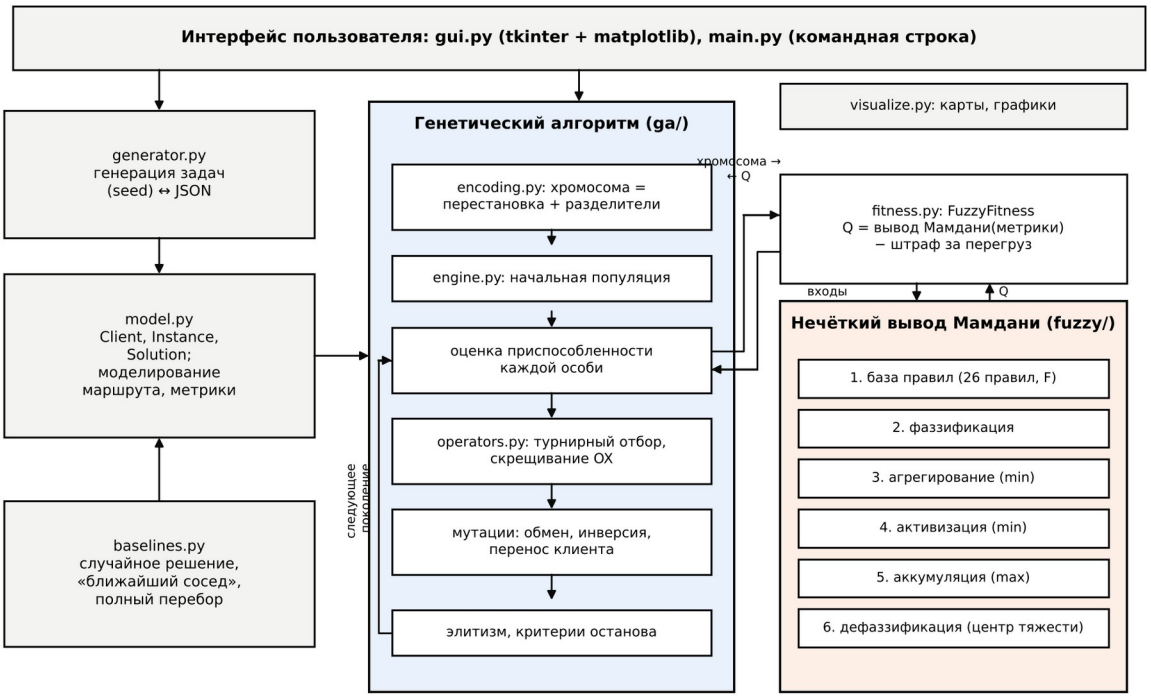



Рисунок 3 – Архитектура гибридной интеллектуальной системы

Назначение модулей:

– model.py – классы предметной области (клиент, задача, решение), моделирование движения машины по маршруту и вычисление показателей плана;

– generator.py – генерация тестовых задач с фиксированным начальным значением генератора случайных чисел, сохранение и загрузка в формате JSON;

– fuzzy/ – функции принадлежности, лингвистические переменные, база правил и шесть этапов вывода Мамдани;

– ga/ – кодирование хромосомы, генетические операторы, функция приспособленности и главный цикл ГА;

– baselines.py – базовые методы для сравнения: случайное решение, «ближайший сосед», полный перебор;

– visualize.py, gui.py, main.py – графики, графический и консольный интерфейсы.

Связь методов показана на рисунке 3 стрелками между блоками: на этапе оценки приспособленности ГА передаёт хромосому функции приспособленности, та вычисляет пять показателей плана и вызывает нечёткий вывод, а полученная оценка Q возвращается в ГА и используется при отборе родителей. Диаграмма основных классов приведена на рисунке 4.

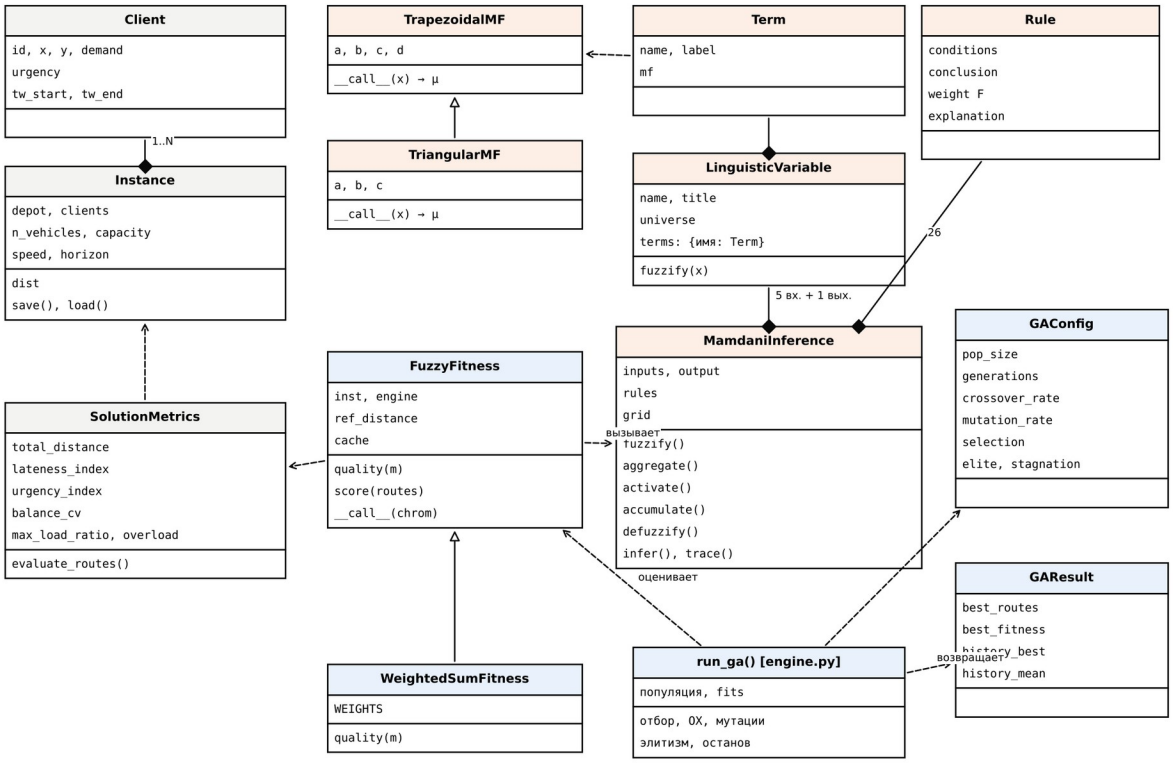



Рисунок 4 – Диаграмма основных классов системы

### 3.2 Модель задачи и генератор исходных данных

Задача описывается классом Instance: координаты склада, список клиентов (класс Client), число машин K, вместимость Q, скорость v = 0,5 км/мин и длительность смены H = 480 мин. Матрица евклидовых расстояний вычисляется один раз и кэшируется. Функция simulate_route моделирует движение машины: время прибытия к очередному клиенту равно времени отъезда от предыдущего плюс время в пути; при раннем прибытии машина ожидает начала окна, при позднем – фиксируется опоздание (формула 3); затем добавляется время разгрузки 10 мин. Функция evaluate_routes по результатам моделирования всех маршрутов вычисляет показатели, служащие входами нечёткой модели.

Генератор generate_instance(n_clients, n_vehicles, seed) размещает клиентов равномерно в квадрате 40 × 40 км со складом в центре, назначает объёмы заказов от 1 до 10 единиц и срочность (60 % обычных, 25 % повышенной срочности, 15 % срочных). Вместимость машин подбирается так, чтобы парк был загружен на 85 %. Чтобы задача заведомо имела решение без опозданий, временные окна строятся вокруг моментов прибытия в скрытом опорном плане, полученном методом заметания: клиенты сортируются по полярному углу и делятся на K секторов, срочные клиенты в секторе объезжаются первыми. Опорный план системе не сообщается, а его поиск – нетривиальная задача. Пример файла задачи приведён в приложении Б.

### 3.3 Нечёткая модель оценки качества плана

#### 3.3.1 Лингвистические переменные

Входные переменные вычисляются для плана в целом, а не для отдельных маршрутов с последующим усреднением. Это обусловлено тем, что сбалансированность по определению характеризует соотношение маршрутов, а суммарные показатели (весь пробег, все опоздания) соответствуют тому, как план оценивает ЛПР. Чёткие значения входов:

– протяжённость – отношение суммарного пробега D к пробегу $D_{ref}$ жадного решения «ближайший сосед» на той же задаче:

$$L = \frac{D}{D_{ref}} \tag{13}$$

– загрузка транспорта – максимальная загрузка машины $W_k$ в долях вместимости:

$$3 = \max_k \frac{W_k}{Q} \tag{14}$$

– соблюдение временных окон – средняя по клиентам относительная величина опоздания, где опоздание на 60 мин и более считается полным:

$$T = \frac{1}{N} \sum_{i=1}^{N} min\left\{1, \frac{late_i}{60}\right\} \tag{15}$$

– обслуживание срочных заказов – средневзвешенное время начала обслуживания срочных клиентов в долях смены (вес $u_i - 1$ равен 0 для обычных, 1 для повышенной срочности и 2 для срочных заказов):

$$U = \frac{\sum_i (u_i - 1) \cdot \frac{s_i}{H}}{\sum_i (u_i - 1)} \tag{16}$$

– сбалансированность – коэффициент вариации длительностей маршрутов $\tau_k$ всех K машин (отношение среднеквадратического отклонения к среднему):

$$B = \frac{\sigma_\tau}{m_\tau} \tag{17}$$

Нормировка протяжённости к эталонному решению делает переменную безразмерной и одинаково интерпретируемой для задач любого размера: L = 1 означает «как у жадного алгоритма», L = 0,9 – на 10 % короче. Выходная переменная «Качество решения» задана на универсуме [0; 1] пятью треугольными термами. Параметры всех функций принадлежности приведены в таблице 6, их графики – на рисунке 5.

Таблица 6 – Лингвистические переменные и функции принадлежности

| Переменная | Универсум | Термы и параметры функций принадлежности |
|---|---|---|
| Протяжённость (L) | [0,6; 2,6] | «короткая» – Trap(0,60; 0,60; 0,80; 1,10); «средняя» – Tri(0,80; 1,10; 1,60); «длинная» – Trap(1,10; 1,60; 2,60; 2,60) |
| Загрузка транспорта (З) | [0,0; 2,0] | «низкая» – Trap(0,00; 0,00; 0,50; 0,80); «нормальная» – Trap(0,50; 0,80; 1,00; 1,10); «перегрузка» – Trap(1,00; 1,10; 2,00; 2,00) |
| Соблюдение временных окон (T) | [0,0; 1,0] | «хорошее» – Tri(0,00; 0,00; 0,12); «удовлетворительное» – Tri(0,00; 0,12; 0,45); «плохое» – Trap(0,12; 0,45; 1,00; 1,00) |
| Обслуживание срочных заказов (U) | [0,0; 1,0] | «хорошее» – Trap(0,00; 0,00; 0,05; 0,25); «среднее» – Tri(0,05; 0,25; 0,50); «плохое» – Trap(0,25; 0,50; 1,00; 1,00) |
| Сбалансированность (B) | [0,0; 1,5] | «равномерная» – Trap(0,00; 0,00; 0,05; 0,40); «неравномерная» – Trap(0,05; 0,40; 1,50; 1,50) |
| Качество решения (Q) | [0,0; 1,0] | «очень низкое» – Tri(0,00; 0,00; 0,25); «низкое» – Tri(0,00; 0,25; 0,50); «среднее» – Tri(0,25; 0,50; 0,75); «высокое» – Tri(0,50; 0,75; 1,00); «очень высокое» – Tri(0,75; 1,00; 1,00) |

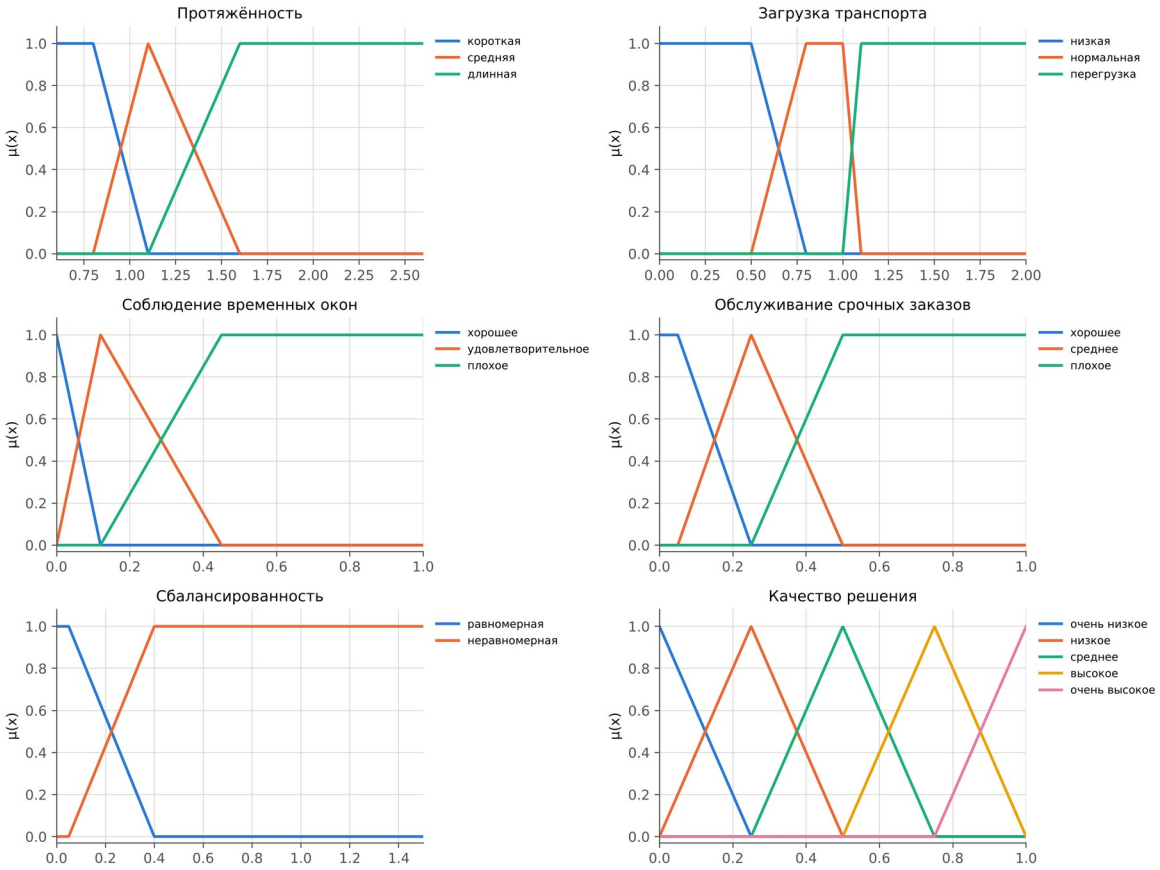



Рисунок 5 – Функции принадлежности лингвистических переменных

Соседние термы каждой входной переменной образуют разбиение Руспини: их склоны пересекаются на уровне 0,5, а сумма степеней принадлежности в любой точке равна единице. Такой выбор сделан по

результатам отладки. В первой версии модели использовались трапеции с широкими плоскими вершинами, и ГА останавливался на «плато»: если все сработавшие правила имеют одно и то же заключение, центр тяжести не зависит от степеней их истинности, и улучшение плана не меняет оценку. При разбиении Руспини почти любое значение принадлежит сразу двум термам, срабатывают правила с разными заключениями, и оценка изменяется плавно.

### 3.3.2 База нечётких правил

База содержит 26 правил с весовыми коэффициентами (таблица 7). Её ядро – правила 1–9 – образуют полную таблицу «Протяжённость × Соблюдение временных окон»: улучшение любого из двух главных показателей на один терм повышает заключение на один терм. Поскольку термы каждой переменной покрывают весь универсум, при любых входных данных срабатывает хотя бы одно из этих правил, и база правил полна. Правила 10–26 уточняют оценку с учётом срочных заказов, загрузки и сбалансированности. Положительные правила для вспомогательных критериев сформулированы только вместе с хорошим значением пробега или окон («ЕСЛИ срочные обслужены хорошо И опозданий нет ТО …»), чтобы хорошая сбалансированность не «вытягивала» план с массовыми опозданиями. Весовые коэффициенты $F < 1$ назначены правилам, в которых эксперт менее уверен или которые описывают второстепенные критерии.

Таблица 7 – База нечётких продукционных правил

| № | Правило | F | Пояснение |
|---|---|---|---|
| 1 | ЕСЛИ Протяжённость = «короткая» И Соблюдение временных окон = «хорошее» ТО Качество решения = «очень высокое» | 1,0 | Короткий пробег без опозданий — главная цель планирования доставки. |
| 2 | ЕСЛИ Протяжённость = «короткая» И Соблюдение временных окон = «удовлетворительное» ТО Качество решения = «высокое» | 1,0 | Пробег экономичен, опоздания единичны и невелики. |
| 3 | ЕСЛИ Протяжённость = «короткая» И Соблюдение временных окон = «плохое» ТО Качество решения = «среднее» | 1,0 | Экономия пробега получена ценой массовых опозданий — компромиссный план. |
| 4 | ЕСЛИ Протяжённость = «средняя» И Соблюдение временных окон = «хорошее» ТО Качество решения = «высокое» | 1,0 | Опозданий нет, перерасход пробега умеренный. |

| № | Правило | F | Пояснение |
|---|---|---|---|
| 5 | ЕСЛИ Протяжённость = «средняя» И Соблюдение временных окон = «удовлетворительное» ТО Качество решения = «среднее» | 1,0 | Средний пробег и отдельные опоздания — план посредственный. |
| 6 | ЕСЛИ Протяжённость = «средняя» И Соблюдение временных окон = «плохое» ТО Качество решения = «низкое» | 1,0 | Пробег не сокращён, а клиенты обслуживаются с опозданием. |
| 7 | ЕСЛИ Протяжённость = «длинная» И Соблюдение временных окон = «хорошее» ТО Качество решения = «среднее» | 1,0 | Окна соблюдены, но за счёт большого перерасхода топлива и времени. |
| 8 | ЕСЛИ Протяжённость = «длинная» И Соблюдение временных окон = «удовлетворительное» ТО Качество решения = «низкое» | 1,0 | Длинные маршруты не спасают от опозданий — план неэффективен. |
| 9 | ЕСЛИ Протяжённость = «длинная» И Соблюдение временных окон = «плохое» ТО Качество решения = «очень низкое» | 1,0 | Большой пробег и массовые опоздания одновременно — план следует отвергнуть. |
| 10 | ЕСЛИ Обслуживание срочных заказов = «хорошее» И Соблюдение временных окон = «хорошее» ТО Качество решения = «очень высокое» | 0,8 | Срочные клиенты обслужены в начале смены, опозданий нет. |
| 11 | ЕСЛИ Обслуживание срочных заказов = «хорошее» И Протяжённость = «короткая» ТО Качество решения = «высокое» | 0,7 | Срочные заказы доставлены первыми без перерасхода пробега. |
| 12 | ЕСЛИ Обслуживание срочных заказов = «среднее» И Соблюдение временных окон = «хорошее» ТО Качество решения = «высокое» | 0,6 | Опозданий нет, срочные заказы доставлены не первыми, но вовремя. |
| 13 | ЕСЛИ Обслуживание срочных заказов = «среднее» И Соблюдение временных окон = «удовлетворительное» ТО Качество решения = «среднее» | 0,6 | Посредственное обслуживание и срочных, и обычных клиентов. |
| 14 | ЕСЛИ Обслуживание срочных заказов = «плохое» ТО Качество решения = «низкое» | 0,8 | Срочные заказы доставляются в конце смены — риск потери клиентов. |
| 15 | ЕСЛИ Обслуживание срочных заказов = «плохое» И Соблюдение временных окон = «плохое» ТО Качество решения = «очень низкое» | 1,0 | Опоздания вместе с поздней доставкой срочных заказов — худший сервис. |
| 16 | ЕСЛИ Загрузка транспорта = «перегрузка» ТО Качество решения = «очень низкое» | 1,0 | Перегруз машины физически недопустим (дополнительно штрафуется вне нечёткого блока). |
| 17 | ЕСЛИ Загрузка транспорта = «нормальная» И Протяжённость = «короткая» И Соблюдение временных окон = «хорошее» ТО Качество решения = «очень высокое» | 0,8 | Рациональная загрузка кузова при коротком пробеге и без опозданий. |
| 18 | ЕСЛИ Загрузка транспорта = «низкая» И Протяжённость = «длинная» ТО Качество решения = «низкое» | 0,8 | Машины недогружены, но много ездят — парк используется неэффективно. |
| 19 | ЕСЛИ Сбалансированность = «равномерная» И Соблюдение временных окон = «хорошее» ТО Качество решения = «высокое» | 0,7 | Нагрузка водителей равномерна, сверхурочных нет, клиенты обслужены вовремя. |
| 20 | ЕСЛИ Сбалансированность = «равномерная» И Протяжённость = «короткая» ТО Качество решения = «высокое» | 0,6 | Равные по длительности маршруты при экономичном пробеге. |
| 21 | ЕСЛИ Сбалансированность = «неравномерная» ТО Качество решения = «низкое» | 0,4 | Один водитель перегружен, другие простаивают. |
| 22 | ЕСЛИ Сбалансированность = «неравномерная» И Соблюдение временных окон = «удовлетворительное» ТО Качество решения = | 0,7 | Перегруженный водитель опаздывает, пока другие простаивают. |

| № | Правило | F | Пояснение |
|---|---|---|---|
| | «низкое» | | |
| 23 | ЕСЛИ Сбалансированность = «неравномерная» И Протяжённость = «средняя» ТО Качество решения = «низкое» | 0,6 | Неравномерная нагрузка при среднем пробеге — признак неудачного распределения заказов. |
| 24 | ЕСЛИ Протяжённость = «короткая» И Обслуживание срочных заказов = «хорошее» И Сбалансированность = «равномерная» ТО Качество решения = «очень высокое» | 0,9 | Экономичный план с приоритетом срочных заказов и равной нагрузкой водителей. |
| 25 | ЕСЛИ Протяжённость = «длинная» И Обслуживание срочных заказов = «плохое» ТО Качество решения = «очень низкое» | 0,8 | Длинные маршруты, на которых срочные заказы доставляются последними. |
| 26 | ЕСЛИ Протяжённость = «средняя» И Обслуживание срочных заказов = «хорошее» ТО Качество решения = «высокое» | 0,6 | Небольшой перерасход пробега оправдан ранней доставкой срочных заказов. |

### 3.3.3 Реализация механизма вывода

Алгоритм вывода реализован классом MamdaniInference в точном соответствии с шестью этапами, описанными в подразделе 2.5 (рисунок 6). Универсум выходной переменной дискретизирован равномерной сеткой из G = 201 точки, поэтому интегралы в формуле центра тяжести 12 заменены суммами:

$$Q \approx \frac{\sum_{g=1}^{G} x_g \cdot \mu_E(x_g)}{\sum_{g=1}^{G} \mu_E(x_g)} \tag{18}$$

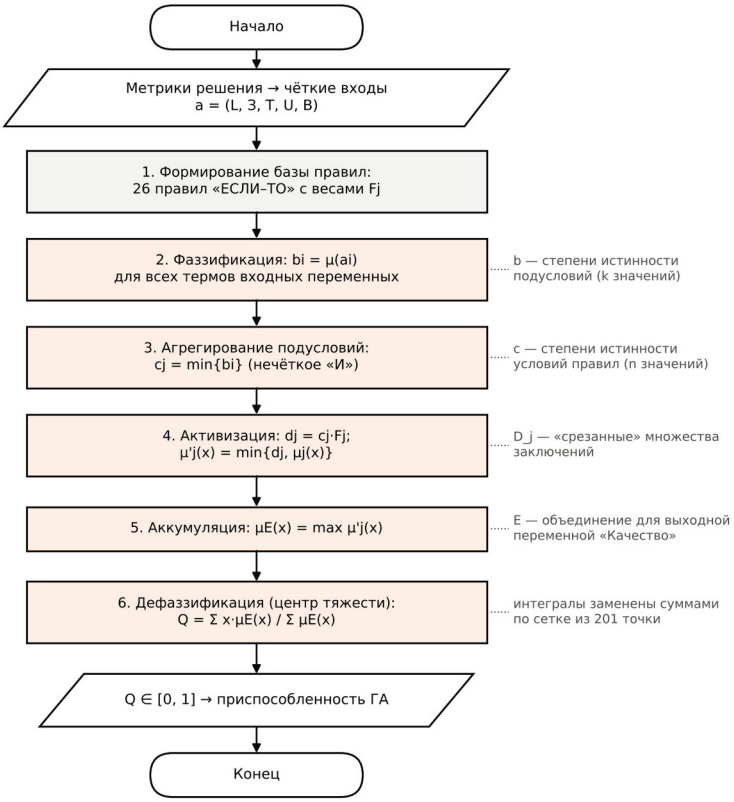



Рисунок 6 – Схема алгоритма нечёткого вывода Мамдани

Функции принадлежности термов-заключений вычисляются на сетке один раз при создании объекта и хранятся в матрице размером n × G; это позволяет выполнить активизацию всех правил одной векторной операцией. Ключевые методы приведены в листинге 1.

Листинг 1 – Этапы агрегирования, активизации, аккумуляции и дефаззификации

```
def aggregate(self, fuzz: dict[str, dict[str, float]]) -> np.ndarray:
    # c_j = min{b_i} по всем подусловиям (переменная v, терм t) правила j
    return np.array([min(fuzz[v][t] for v, t in r.conditions) for r in
self.rules])

def activate(self, c: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    d = c * self._weights                                    # d_j = c_j · F_j
    # μ'_j(x) = min(d_j, μ_j(x)): строка j — «срезанный» терм заключения
правила j
    activated = np.minimum(d[:, None], self._concl_mf)
    return d, activated

@staticmethod
def accumulate(activated: np.ndarray) -> np.ndarray:
    return activated.max(axis=0)                             # μ_E(x) = max_j
μ'_j(x)

def defuzzify(self, mu: np.ndarray) -> float:
    area = mu.sum()                                          # Σ μ_E(x_g)
```

```
    if area <= 1e-12:
        return 0.0
    return float((self.grid * mu).sum() / area)            # Σ
x_g·μ_E(x_g) / Σ μ_E(x_g)
```

Метод trace сохраняет все промежуточные результаты (степени принадлежности, $c_j$, $d_j$, аккумулированное множество), что позволяет показать ЛПР, какие правила определили оценку (подраздел 4.6).

## 3.4 Генетический алгоритм

### 3.4.1 Кодирование решения

Решение кодируется перестановкой длины N + K − 1: гены 1, …, N – номера клиентов, гены N + 1, …, N + K − 1 – разделители маршрутов (рисунок 7). Хромосома читается слева направо; разделитель означает переход к следующей машине. Одна хромосома одновременно задаёт и распределение заказов по машинам, и порядок объезда. Альтернативное кодирование «гигантский тур с последующим разбиением по вместимости» [12] было отклонено: оно само гарантирует допустимость по вместимости, но может потребовать больше K машин и лишает смысла переменную «Загрузка». Перестановочное кодирование позволяет использовать стандартный оператор OX без доработки: потомок OX – всегда корректная перестановка.

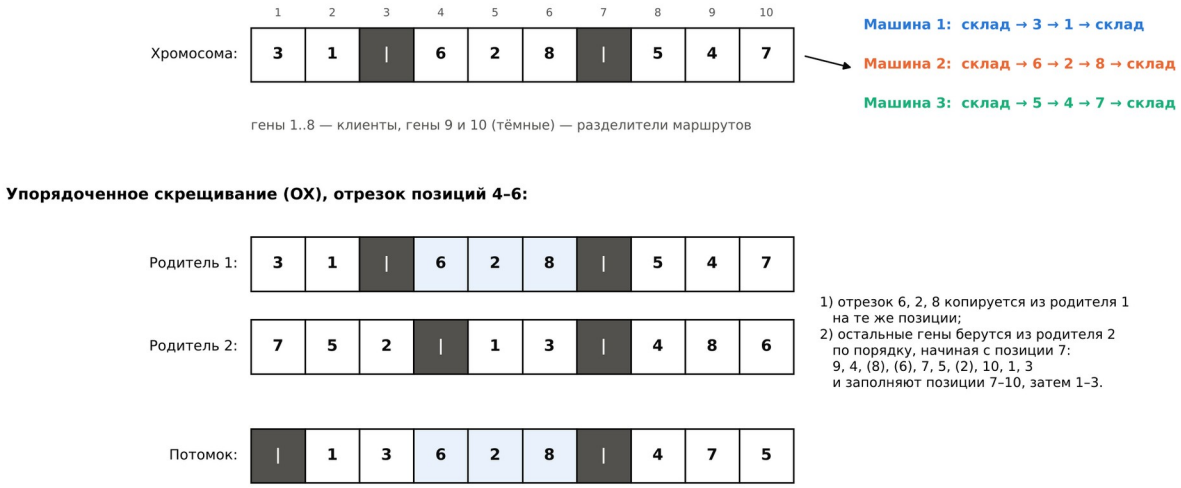



Рисунок 7 – Кодирование плана доставки и упорядоченное скрещивание

### 3.4.2 Функция приспособленности

Приспособленность особи x равна нечёткой оценке её качества за вычетом штрафа за перегруз:

$$f(x)=Q(x)-P(x),P(x)=\begin{cases}0,\text{если перегруза нет;}\\ 1+\dfrac{\Delta}{Q},\text{если }\Delta>0,\end{cases} \quad (19)$$

где Δ – суммарное превышение вместимости по всем машинам. Поскольку Q ∈ [0; 1], любой план с перегрузом получает отрицательную приспособленность и оказывается хуже любого допустимого плана; слагаемое Δ/Q сохраняет различия между недопустимыми планами, чтобы ГА мог постепенно выводить популяцию из недопустимой области. Вычисленные значения кэшируются по хромосоме: одинаковые особи, часто возникающие после элитизма и скрещивания похожих родителей, повторно не оцениваются (листинг 2).

Листинг 2 – Нечёткая функция приспособленности

```
def overload_penalty(m: SolutionMetrics, capacity: int) -> float:
    if m.overload <= 0:
        return 0.0
    return OVERLOAD_BASE + OVERLOAD_RATE * m.overload / capacity

def score(self, routes: list[list[int]]) -> float:
    m = evaluate_routes(routes, self.inst)
    return self.quality(m) - overload_penalty(m, self.inst.capacity)

def __call__(self, chrom) -> float:
    from src.ga.encoding import decode
    key = tuple(chrom)
    val = self.cache.get(key)
    if val is None:                    # особь ещё не оценивалась
        self.evaluations += 1
        # хромосома → маршруты → метрики → нечёткий вывод Мамдани − штраф
        val = self.score(decode(chrom, self.inst.n_clients))
        self.cache[key] = val
    return val
```

### 3.4.3 Генетические операторы

Отбор родителей – турнирный с размером турнира k = 3 (для сравнения реализованы также рулеточный и ранговый отбор). Скрещивание OX выполняется с вероятностью $p_c$ = 0,9 и даёт двух потомков (родители

меняются ролями); реализация приведена в листинге 3. Каждый потомок с вероятностью $p_m$ подвергается одной из трёх мутаций, выбранной равновероятно:

– обмен двух случайных генов – меняет местами клиентов (возможно, из разных маршрутов) или перемещает разделитель;

– инверсия отрезка – разворачивает часть маршрута, что устраняет самопересечения (аналог улучшения 2-opt);

– перенос клиента – удаляет случайного клиента из его маршрута и вставляет в случайную позицию другого маршрута, изменяя распределение заказов.

Листинг 3 – Оператор упорядоченного скрещивания OX

```
def order_crossover(p1: list[int], p2: list[int], rng: np.random.Generator)
-> list[int]:
    n = len(p1)
    i, j = sorted(rng.choice(n + 1, size=2, replace=False))
    child = [None] * n
    child[i:j] = p1[i:j]
    taken = set(p1[i:j])
    fill = [p2[(j + k) % n] for k in range(n) if p2[(j + k) % n] not in
taken]
    pos = j % n
    for g in fill:
        while child[pos] is not None:
            pos = (pos + 1) % n
        child[pos] = g
        pos = (pos + 1) % n
    return child
```

### 3.4.4 Начальная популяция, элитизм и останов

Схема алгоритма приведена на рисунке 8. Начальная популяция на 90 % состоит из случайных перестановок и на 10 % – из решений рандомизированного «ближайшего соседа» (следующий клиент выбирается случайно среди трёх ближайших). Такой «засев» популяции эвристикой – распространённый приём при решении задач маршрутизации [12]: без него при N = 50 все случайные особи попадают в область, где все входные переменные имеют худшие термы и нечёткая оценка постоянна. Две лучшие особи каждого поколения переносятся в следующее без изменений (элитизм), поэтому лучшая приспособленность не убывает. Алгоритм останавливается

по достижении заданного числа поколений, при отсутствии улучшения в течение 120 поколений (стагнация) или по лимиту времени. Значения параметров по умолчанию ($P = 100$, $p_c = 0{,}9$, $p_m = 0{,}2$, $k = 3$, две элитные особи) выбраны по результатам экспериментов подраздела 4.4.

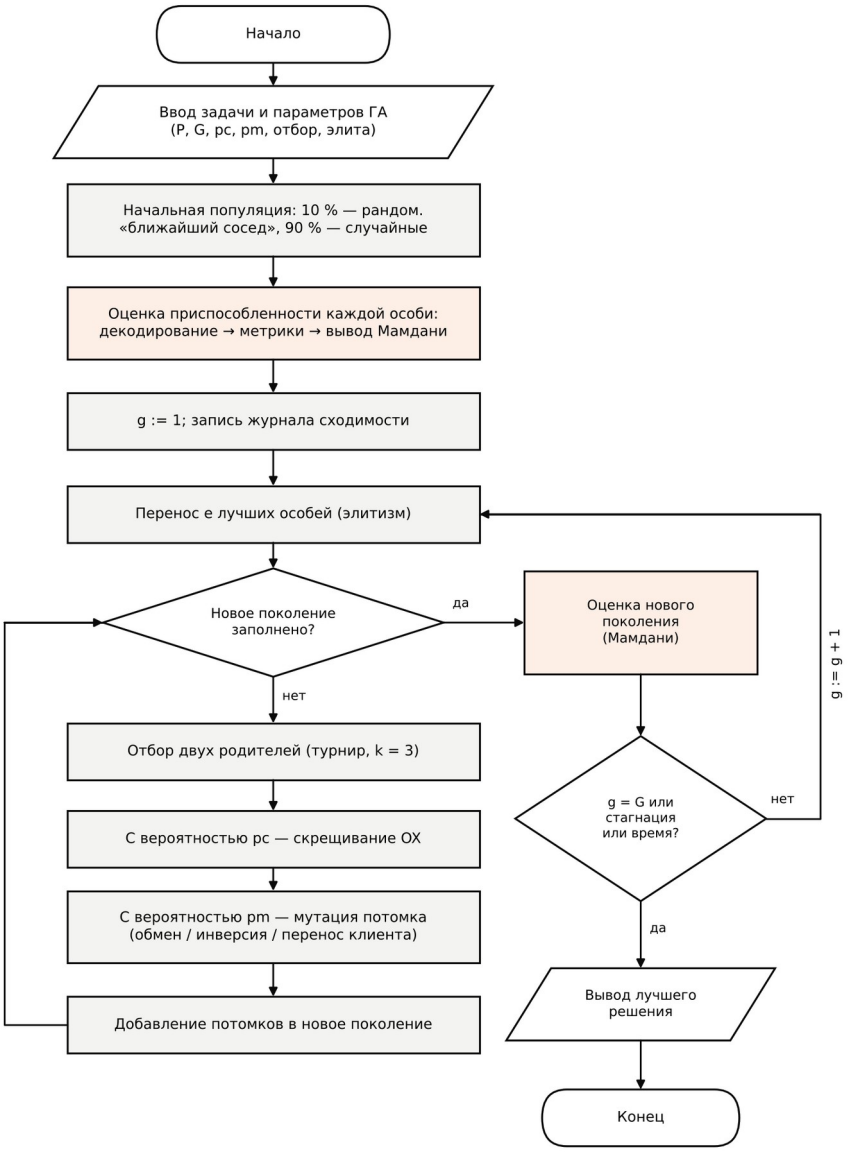



Рисунок 8 – Схема генетического алгоритма

### 3.5 Пользовательский интерфейс

Графический интерфейс (модуль gui.py, библиотека tkinter) содержит панель параметров задачи и ГА, кнопки «Сгенерировать», «Загрузить JSON», «Сохранить JSON», «Решить», «Стоп», «Жадный алгоритм» и три вкладки: карта маршрутов, график сходимости и результат нечёткого вывода. ГА

выполняется в отдельном потоке и каждые десять поколений обновляет карту лучшего решения, поэтому пользователь наблюдает процесс эволюции. По завершении в журнал выводятся показатели плана, маршруты машин и протокол нечёткого вывода: степени принадлежности входов и наиболее сильные сработавшие правила. Консольный интерфейс (модуль main.py) предоставляет те же возможности для пакетного запуска; пример его вывода приведён в приложении В.

## 4 Экспериментальный анализ

### 4.1 Методика экспериментов

Все эксперименты выполняются скриптом experiments/run_experiments.py одной командой; полный прогон занимает около 87 с на четырёхъядерном процессоре (запуски распараллелены по процессам). Задачи генерируются с фиксированными значениями seed и сохраняются в каталоге experiments/instances, а сами алгоритмы используют собственные фиксированные seed, поэтому повторный запуск даёт те же значения приспособленности (время работы, естественно, может немного отличаться).

Для каждого размера N ∈ {10, 20, 30, 50} сгенерировано пять задач (число машин K = 2, 3, 4 и 7 соответственно). Сравниваются четыре метода:

– случайное решение – средние показатели 20 случайных хромосом;

– жадный алгоритм «ближайший сосед» с учётом вместимости;

– ГА со взвешенной суммой тех же нормированных показателей вместо нечёткого блока (веса 0,35; 0,30; 0,20; 0,15 для протяжённости, окон, срочности и сбалансированности) и тем же штрафом за перегруз:

$$Q_{ws} = 1 - \left(0{,}35 \cdot s_L + 0{,}30 \cdot s_T + 0{,}20 \cdot s_U + 0{,}15 \cdot s_B\right) \qquad (20)$$

– предлагаемый ГА с нечёткой оценкой (P = 100, 300 поколений при N ≤ 30 и 500 при N = 50, останов по стагнации 150 поколений).

Для сопоставимости все итоговые планы оцениваются одной и той же нечёткой моделью Q, а также «сырыми» показателями: пробегом, числом опозданий, суммарным опозданием, перегрузом, сбалансированностью B и индексом срочности U.

### 4.2 Сравнение с базовыми методами

Результаты (средние по пяти задачам) приведены в таблице 8 и на рисунке 9.

Таблица 8 – Сравнение методов (среднее ± СКО по пяти задачам)

| N | Метод | Пробег, км | Число опозданий | Опоздание, мин | Перегруз | B | U | Оценка Q | Время, с |
|---|---|---|---|---|---|---|---|---|---|
| 10 | Случайное | 221 ± 20 | 5,5 | 859 | 11,0 | 0,49 | 0,40 | 0,263 ± 0,020 | 0,00 |
| 10 | Ближайший сосед | 169 ± 21 | 2,0 | 190 | 0,0 | 0,16 | 0,19 | 0,548 ± 0,105 | 0,00 |
| 10 | ГА + взвеш. сумма | 158 ± 13 | 0,2 | 13 | 0,0 | 0,06 | 0,13 | 0,736 ± 0,054 | 0,35 |
| 10 | ГА + нечёткая оценка | 164 ± 24 | 0,0 | 0 | 0,0 | 0,03 | 0,13 | 0,780 ± 0,009 | 0,37 |
| 20 | Случайное | 438 ± 30 | 13,7 | 3114 | 25,3 | 0,62 | 0,52 | 0,201 ± 0,002 | 0,00 |
| 20 | Ближайший сосед | 247 ± 25 | 7,4 | 417 | 0,0 | 0,14 | 0,21 | 0,471 ± 0,043 | 0,00 |
| 20 | ГА + взвеш. сумма | 224 ± 26 | 2,6 | 34 | 0,0 | 0,09 | 0,14 | 0,680 ± 0,052 | 0,74 |
| 20 | ГА + нечёткая оценка | 237 ± 25 | 2,4 | 138 | 0,0 | 0,06 | 0,17 | 0,684 ± 0,077 | 0,92 |
| 30 | Случайное | 644 ± 33 | 21,3 | 6421 | 47,6 | 0,73 | 0,58 | 0,203 ± 0,001 | 0,00 |
| 30 | Ближайший сосед | 328 ± 29 | 13,6 | 1046 | 0,0 | 0,30 | 0,28 | 0,428 ± 0,053 | 0,00 |
| 30 | ГА + взвеш. сумма | 285 ± 15 | 4,2 | 64 | 0,0 | 0,05 | 0,14 | 0,712 ± 0,046 | 1,01 |
| 30 | ГА + нечёткая оценка | 313 ± 24 | 4,8 | 138 | 0,0 | 0,05 | 0,14 | 0,720 ± 0,037 | 1,40 |
| 50 | Случайное | 1097 ± 31 | 35,6 | 12033 | 81,8 | 0,84 | 0,59 | 0,204 ± 0,001 | 0,00 |
| 50 | Ближайший сосед | 461 ± 22 | 20,6 | 1824 | 0,0 | 0,49 | 0,29 | 0,438 ± 0,070 | 0,00 |
| 50 | ГА + взвеш. сумма | 442 ± 20 | 4,4 | 79 | 0,0 | 0,10 | 0,14 | 0,671 ± 0,027 | 1,97 |
| 50 | ГА + нечёткая оценка | 517 ± 8 | 5,4 | 96 | 0,0 | 0,08 | 0,14 | 0,683 ± 0,044 | 2,45 |

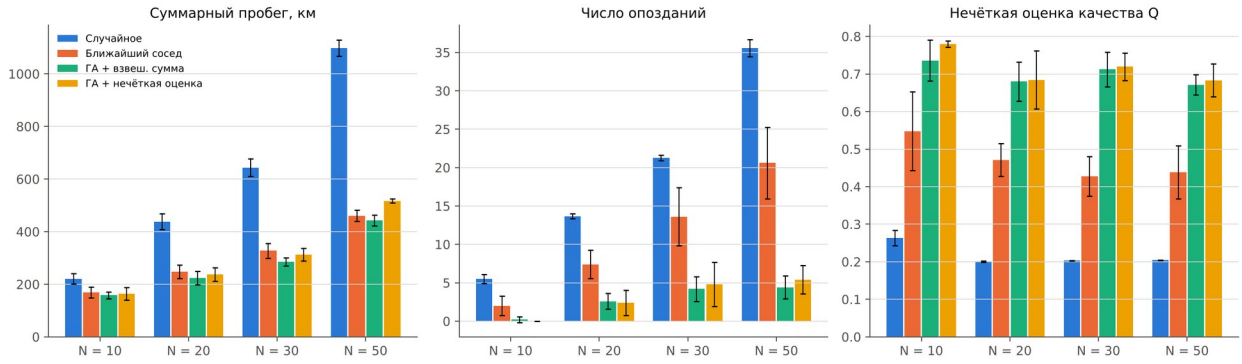



Рисунок 9 – Пробег, число опозданий и нечёткая оценка качества для четырёх методов

Выводы по эксперименту:

– оба варианта ГА многократно превосходят случайное решение и жадный алгоритм по соблюдению временных окон: при N = 50 суммарное опоздание сокращается с 1824 мин у «ближайшего соседа» до 96 мин у ГА с нечёткой оценкой (на 95 %), число опоздавших клиентов – с 20,6 до 5,4;

– при N ≤ 30 ГА с нечёткой оценкой одновременно сокращает и пробег (на 3–5 % относительно жадного алгоритма), то есть строит планы, лучшие по всем критериям; при N = 50 пробег возрастает на 12,2 % – это цена за практически полное устранение опозданий и выравнивание нагрузки водителей (B = 0,08 против 0,49);

– нечёткая оценка качества вырастает на 0,21–0,29 по сравнению с жадным алгоритмом; ни одно решение ГА не содержит перегруза, т. е. штрафная функция надёжно удерживает поиск в допустимой области;

– ГА со взвешенной суммой даёт сопоставимые результаты: по нечёткой оценке Q он немного уступает (например, 0,671 против 0,683 при N = 50), а по пробегу выигрывает, поскольку вес пробега в свёртке наибольший. Различие отражает разные «предпочтения» двух моделей оценки: нечёткая модель по правилам 19–24 сильнее ценит равномерную нагрузку водителей и не позволяет экономии пробега компенсировать опоздания. Преимущество нечёткой модели, таким образом, не в «более высоком числе», а в том, что её предпочтения заданы явными, проверяемыми экспертом правилами, а оценка объясняется протоколом вывода (подраздел 4.6);

– время работы ГА – от 0,37 с при N = 10 до 2,45 с при N = 50, что приемлемо для ежедневного планирования.

### 4.3 Сравнение с полным перебором

Для проверки того, насколько близко ГА подходит к оптимуму, сгенерированы задачи с N = 5…8 клиентами и K = 2 машинами, для которых функция приспособленности максимизирована полным перебором всех различных планов (с точностью до перенумерации машин). ГА с параметрами по умолчанию запускался десять раз с разными seed (таблица 9).

Таблица 9 – Сравнение ГА с полным перебором (10 запусков ГА)

| N | Число решений | Оптимум f* | Время перебора, с | ГА: средняя f | ГА: худшая f | Найден оптимум, % | Отставание, % | Уникальных оценок ГА | Время ГА, с |
|---|---|---|---|---|---|---|---|---|---|
| 5 | 360 | 0,7931 | 0,02 | 0,7931 | 0,7931 | 100 | 0,0 | 297 | 0,29 |
| 6 | 2 520 | 0,6720 | 0,13 | 0,6720 | 0,6720 | 100 | 0,0 | 651 | 0,34 |
| 7 | 20 160 | 0,7692 | 1,30 | 0,7508 | 0,6939 | 60 | 2,4 | 818 | 0,29 |
| 8 | 181 440 | 0,7049 | 11,15 | 0,6924 | 0,6145 | 80 | 1,8 | 1223 | 0,36 |

При N = 5 и 6 ГА находит точный оптимум во всех запусках. При N = 7 и 8 оптимум найден в 60 % и 80 % запусков соответственно, а среднее отставание от оптимума составляет 2,4 % и 1,8 %. При N = 8 ГА вычисляет приспособленность лишь 1223 уникальных особей – 0,7 % от 181 440 вариантов полного перебора – и работает примерно в 31 раз быстрее перебора; с ростом N это соотношение быстро растёт в пользу ГА (таблица 2). Эксперимент подтверждает теоретическое положение подраздела 2.4: ГА находит оптимальное или близкое к нему решение, но не гарантирует оптимальности в каждом запуске. На практике для повышения надёжности ГА можно запускать несколько раз с разными seed и выбирать лучший результат.

### 4.4 Сходимость генетического алгоритма

На рисунке 10 показана динамика лучшей и средней приспособленности в популяции для задачи N = 30 (seed 1). В начальной популяции средняя приспособленность отрицательна: большинство случайных особей перегружены. Уже за 10–15 поколений средняя приспособленность становится положительной – популяция переходит в допустимую область. Лучшая приспособленность растёт ступенчато от 0,342 (лучшая особь начальной популяции, полученная «засевом») до 0,772, причём к 50-му поколению достигается 0,553, а дальнейший рост связан с устранением последних опозданий и выравниванием маршрутов. Разрыв между лучшей и средней приспособленностью сохраняется до конца – мутации поддерживают разнообразие популяции, и преждевременной

сходимости не наблюдается. На рисунке 11 приведены кривые лучшей приспособленности для пяти запусков с разными seed: итоговые значения различаются, что отражает стохастическую природу ГА.

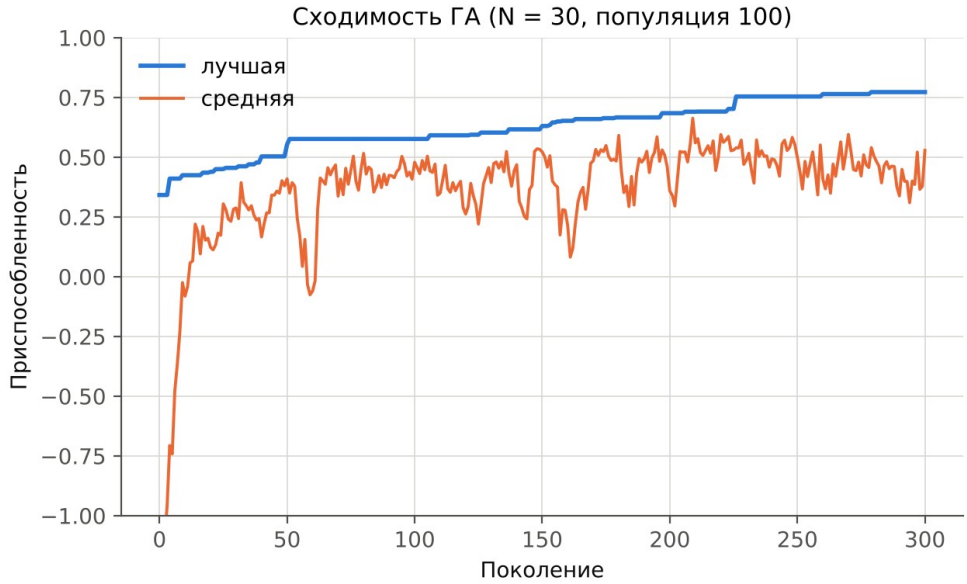



Рисунок 10 – Сходимость ГА: лучшая и средняя приспособленность (N = 30)




Рисунок 11 – Лучшая приспособленность в пяти запусках ГА с разными seed (N = 30)

### 4.5 Влияние параметров генетического алгоритма

Исследовано влияние размера популяции, вероятности мутации и типа отбора на итоговую приспособленность при фиксированном числе поколений (300, без останова по стагнации). Каждая конфигурация запускалась на

восьми задачах с N = 30; остальные параметры – по умолчанию. Результаты приведены в таблице 10 и на рисунке 12.

Таблица 10 – Влияние параметров ГА на итоговую приспособленность (8 запусков, N = 30)

| Параметр | Значение | Средняя f | СКО | Мин. | Макс. | Время, с | Уникальных оценок |
|---|---|---|---|---|---|---|---|
| Размер популяции | 20 | 0,602 | 0,068 | 0,516 | 0,723 | 0,26 | 1343 |
| Размер популяции | 50 | 0,639 | 0,063 | 0,555 | 0,753 | 0,63 | 3666 |
| Размер популяции | 100 | 0,716 | 0,039 | 0,671 | 0,772 | 1,28 | 8507 |
| Размер популяции | 200 | 0,676 | 0,054 | 0,623 | 0,775 | 2,88 | 20449 |
| Вероятность мутации | 0.01 | 0,523 | 0,052 | 0,447 | 0,583 | 0,77 | 1972 |
| Вероятность мутации | 0.05 | 0,672 | 0,060 | 0,585 | 0,771 | 0,91 | 3616 |
| Вероятность мутации | 0.1 | 0,693 | 0,057 | 0,611 | 0,771 | 1,03 | 5158 |
| Вероятность мутации | 0.2 | 0,716 | 0,039 | 0,671 | 0,772 | 1,33 | 8507 |
| Вероятность мутации | 0.3 | 0,676 | 0,054 | 0,594 | 0,778 | 1,65 | 12607 |
| Тип отбора | турнирный | 0,716 | 0,039 | 0,671 | 0,772 | 1,33 | 8507 |
| Тип отбора | рулетка | 0,558 | 0,053 | 0,471 | 0,631 | 2,91 | 25474 |
| Тип отбора | ранговый | 0,481 | 0,058 | 0,377 | 0,569 | 3,15 | 26072 |

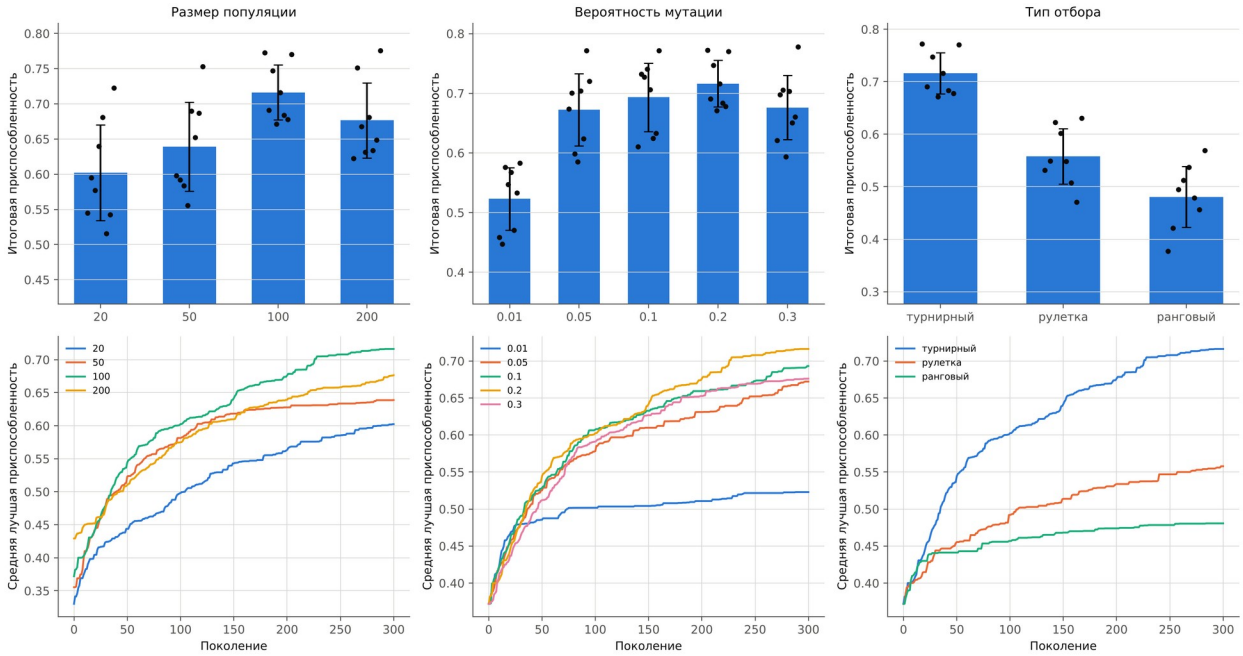



Рисунок 12 – Влияние параметров ГА: итоговая приспособленность (вверху) и средние кривые сходимости (внизу)

Выводы по эксперименту:

– **размер популяции**: при P = 20 и 50 популяции не хватает разнообразия (0,602 и 0,639); наилучший результат получен при P = 100 (0,716). Популяция P = 200 при том же числе поколений даёт несколько меньшее среднее значение (0,676): отбор в большой популяции распространяет лучшие решения медленнее, и за 300 поколений поиск не успевает завершиться, а время работы удваивается. Значение P = 100 выбрано как компромисс качества и времени;

– **вероятность мутации**: при $p_m$ = 0,01 алгоритм быстро теряет разнообразие и останавливается на 0,523 – это классическая преждевременная сходимость; с ростом $p_m$ результат улучшается до 0,716 при $p_m$ = 0,2, а при $p_m$ = 0,3 снова снижается (0,676): слишком частые мутации разрушают найденные удачные фрагменты маршрутов;

– **тип отбора**: турнирный отбор (0,716) значительно превосходит рулеточный (0,558) и ранговый (0,481). Приспособленности допустимых решений лежат в узком диапазоне 0,4–0,8, поэтому после сдвига на минимум рулетка выбирает родителей почти равновероятно, а линейный ранговый отбор при P = 100 также создаёт слабое давление отбора. Турнир же всегда выбирает лучшую из k особей независимо от масштаба значений.

Разброс результатов между задачами (СКО 0,04–0,07) сопоставим с разницей между близкими конфигурациями, поэтому выводы относятся к тенденциям, а не к третьему знаку после запятой.

### 4.6 Пример работы системы и протокол нечёткого вывода

На рисунке 13 показаны карты маршрутов для задачи N = 30: лучшая особь начального поколения, решение жадного алгоритма и итоговое решение ГА. Красными кольцами отмечены клиенты, обслуженные с опозданием. Жадный алгоритм строит компактные маршруты, но, объезжая ближайших клиентов, опаздывает к 11 из них; итоговое решение ГА короче (пробег 273,6 км против 304,4 км) и содержит одно опоздание на 0,3 мин.

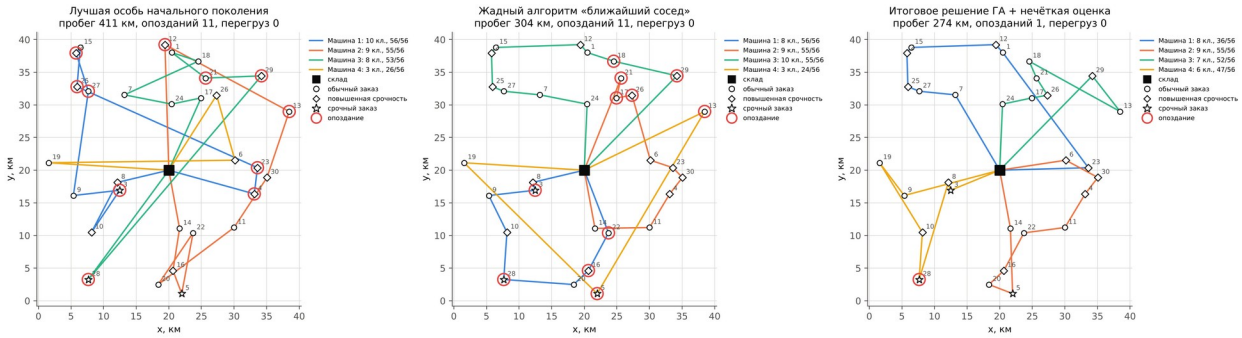



Рисунок 13 – Карты маршрутов: начальное поколение, жадный алгоритм и итоговое решение ГА (N = 30)

Методические указания требуют показывать, каким образом интеллектуальная система пришла к заключению. Для итогового решения рисунка 13 входные показатели равны: L = 0,899, З = 0,982, Т = 0,000, U = 0,197, В = 0,060. Результат фаззификации приведён в таблице 11: например, протяжённость 0,899 принадлежит терму «короткая» со степенью 0,671 и терму «средняя» со степенью 0,329.

Таблица 11 – Фаззификация входных переменных для итогового решения

| **Входная переменная** | **Чёткое значение a** | **Степени принадлежности b = μ(a)** |
|---|---|---|
| Протяжённость | 0,899 | «короткая» = 0,671; «средняя» = 0,329; «длинная» = 0,000 |
| Загрузка транспорта | 0,982 | «низкая» = 0,000; «нормальная» = 1,000; «перегрузка» = 0,000 |
| Соблюдение временных окон | 0,000 | «хорошее» = 0,999; «удовлетворительное» = 0,001; «плохое» = 0,000 |
| Обслуживание срочных заказов | 0,197 | «хорошее» = 0,263; «среднее» = 0,737; «плохое» = 0,000 |
| Сбалансированность | 0,060 | «равномерная» = 0,973; «неравномерная» = 0,027 |

В таблице 12 перечислены правила с ненулевой степенью истинности, их подусловия, результат агрегирования $c_j$ (минимум степеней подусловий), весовой коэффициент и степень истинности заключения $d_j = c_j \cdot F_j$. Наибольший вклад вносят правила П19 (d = 0,681), П1 (d = 0,671) и П17 (d = 0,537).

Таблица 12 – Сработавшие правила и степени истинности

| **Правило** | **Подусловия и их степени истинности b** | **c = min b** | **F** | **d = c·F** | **Заключение (терм «Качества»)** |
|---|---|---|---|---|---|
| П1 | L=короткая: 0,671; T=хорошее: 0,999 | 0,671 | 1,0 | 0,671 | очень высокое |
| П2 | L=короткая: 0,671; T=удовлетворительное: 0,001 | 0,001 | 1,0 | 0,001 | высокое |
| П4 | L=средняя: 0,329; T=хорошее: 0,999 | 0,329 | 1,0 | 0,329 | высокое |
| П5 | L=средняя: 0,329; T=удовлетворительное: 0,001 | 0,001 | 1,0 | 0,001 | среднее |
| П10 | U=хорошее: 0,263; T=хорошее: 0,999 | 0,263 | 0,8 | 0,210 | очень высокое |
| П11 | U=хорошее: 0,263; L=короткая: 0,671 | 0,263 | 0,7 | 0,184 | высокое |
| П12 | U=среднее: 0,737; T=хорошее: 0,999 | 0,737 | 0,6 | 0,442 | высокое |
| П13 | U=среднее: 0,737; T=удовлетворительное: 0,001 | 0,001 | 0,6 | 0,001 | среднее |
| П17 | З=нормальная: 1,000; L=короткая: 0,671; T=хорошее: 0,999 | 0,671 | 0,8 | 0,537 | очень высокое |
| П19 | B=равномерная: 0,973; T=хорошее: 0,999 | 0,973 | 0,7 | 0,681 | высокое |
| П20 | B=равномерная: 0,973; L=короткая: 0,671 | 0,671 | 0,6 | 0,403 | высокое |
| П21 | B=неравномерная: 0,027 | 0,028 | 0,4 | 0,011 | низкое |
| П22 | B=неравномерная: 0,027; T=удовлетворительное: 0,001 | 0,001 | 0,7 | 0,001 | низкое |
| П23 | B=неравномерная: 0,027; L=средняя: 0,329 | 0,028 | 0,6 | 0,017 | низкое |
| П24 | L=короткая: 0,671; U=хорошее: 0,263; B=равномерная: 0,973 | 0,263 | 0,9 | 0,237 | очень высокое |
| П26 | L=средняя: 0,329; U=хорошее: 0,263 | 0,263 | 0,6 | 0,158 | высокое |

После аккумуляции каждый терм выходной переменной «срезан» на уровне максимальной степени истинности правил с этим заключением (таблица 13). Результирующее нечёткое множество и его центр тяжести показаны на рисунке 14: множество сосредоточено на термах «высокое» и «очень высокое», а дефаззификация даёт Q = 0,772. Таким образом, ЛПР получает не только число, но и обоснование: план оценён высоко потому, что он короче эталонного, не имеет существенных опозданий и равномерно загружает водителей, а оценку несколько снижает то, что срочные заказы доставлены не в самом начале смены (U = 0,197 – терм «среднее» со степенью 0,737).

Таблица 13 – Результат аккумуляции и дефаззификации

| **Терм выходной переменной** | **Уровень среза max d_j** |
|---|---|
| очень низкое | 0,000 |

| Терм выходной переменной | Уровень среза max d_j |
|---|---|
| низкое | 0,016 |
| среднее | 0,001 |
| высокое | 0,681 |
| очень высокое | 0,671 |
| Результат дефаззификации Q | 0,7724 |

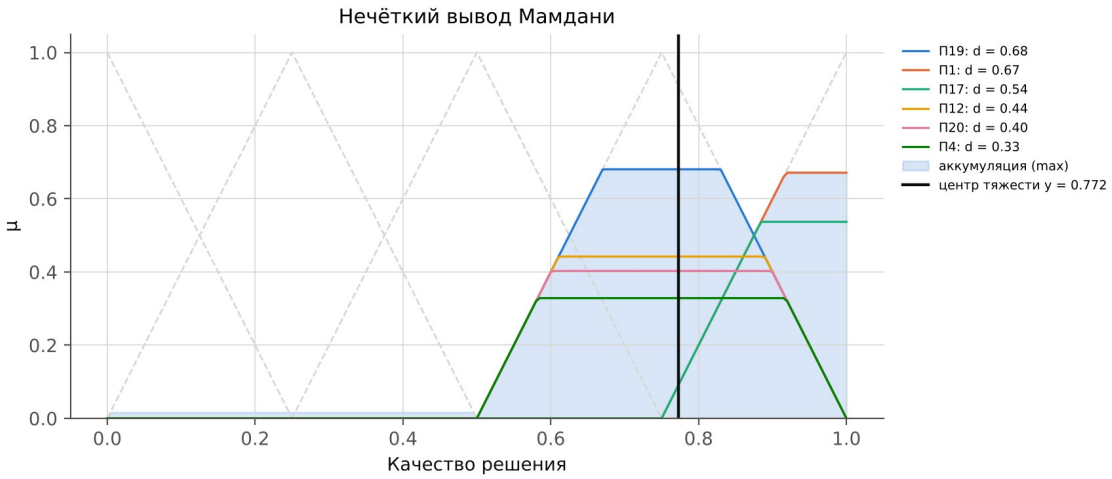



Рисунок 14 – Активизированные множества, аккумуляция и центр тяжести для итогового решения

### 4.7 Работа программы

Ниже отведено место для снимков экрана главного окна программы: панели параметров, карты маршрутов, графика сходимости и вкладки нечёткого вывода.

**[СКРИНШОТ: окно программы с решённой задачей на 30 клиентов (вкладка «Маршруты»)]**

**[СКРИНШОТ: вкладка «Сходимость» после завершения работы ГА]**

**[СКРИНШОТ: вкладка «Нечёткий вывод» и журнал с протоколом сработавших правил]**

Эксперименты подтверждают работоспособность гибридной системы: ГА с нечёткой функцией приспособленности устойчиво строит допустимые планы, практически устраняющие опоздания, на малых задачах находит

точный оптимум, а нечёткая модель даёт интерпретируемое обоснование оценки каждого плана.

# ЗАКЛЮЧЕНИЕ

Проделанная в рамках курсовой работы деятельность позволила ответить на поставленные вопросы, цели и задачи исследования. Основными задачами исследования являлись:

– анализ предметной области планирования доставки «последней мили», постановка задачи маршрутизации транспорта с ограничением вместимости и временными окнами, выявление критериев качества плана и зависимостей, которые должен учитывать ЛПР;

– обзор подходов искусственного интеллекта и обоснование выбора генетического алгоритма для поиска плана и нечёткой логики для его оценки, а также гибридной схемы, в которой нечёткий вывод служит функцией приспособленности ГА;

– разработка нечёткой модели оценки качества плана и генетического алгоритма;

– программная реализация интеллектуальной системы с графическим и консольным интерфейсами;

– экспериментальный анализ системы.

Для решения этих задач были применены следующие методы:

– генетический алгоритм с перестановочным кодированием плана (клиенты и разделители маршрутов), турнирным отбором, упорядоченным скрещиванием OX, мутациями обмена, инверсии и переноса клиента, элитизмом и критериями останова по числу поколений и стагнации;

– нечёткая логика: пять входных и одна выходная лингвистические переменные с треугольными и трапециевидными функциями принадлежности, база из 26 продукционных правил с весовыми коэффициентами, алгоритм вывода Мамдани (агрегирование min, активизация min, аккумуляция max, дефаззификация по центру тяжести);

– имитационное моделирование движения транспорта по маршрутам, штрафные функции для жёстких ограничений, вычислительный эксперимент с сравнением методов и статистической обработкой результатов.

В итоге выполнения курсовой работы удалось получить следующий результат решения конкретной проблемы в рамках выбранной предметной области: разработана гибридная интеллектуальная система планирования маршрутов доставки, в которой генетический алгоритм ищет распределение заказов по машинам и порядок объезда, а нечёткий вывод Мамдани на каждом поколении оценивает каждую особь. Система строит допустимые по вместимости планы за 0,4–2,5 с; на задачах с 50 клиентами она сокращает суммарное опоздание по сравнению с жадным алгоритмом «ближайший сосед» на 95 %, а на задачах с 10–30 клиентами одновременно сокращает и пробег на 3–5 %. На малых задачах ($N \leq 6$) система во всех запусках находит точный оптимум, подтверждённый полным перебором, а при $N = 7…8$ среднее отставание от оптимума не превышает 2,5 %. Экспериментально обоснован выбор параметров ГА (популяция 100, вероятность мутации 0,2, турнирный отбор). Каждая оценка плана сопровождается протоколом нечёткого вывода, объясняющим ЛПР, какие правила и с какой степенью истинности определили результат. Система может применяться диспетчером службы доставки как средство поддержки принятия решений, а её база правил – дополняться и уточняться экспертом без изменения программного кода.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Lenstra, J. K. Complexity of vehicle routing and scheduling problems / J. K. Lenstra, A. H. G. Rinnooy Kan // Networks. – 1981. – Vol. 11, № 2. – P. 221–227.

2. Goldberg, D. E. Genetic Algorithms in Search, Optimization, and Machine Learning / D. E. Goldberg. – Reading : Addison-Wesley, 1989. – 412 p.

3. Гладков, Л. А. Генетические алгоритмы / Л. А. Гладков, В. В. Курейчик, В. М. Курейчик ; под ред. В. М. Курейчика. – 2-е изд., испр. и доп. – М. : Физматлит, 2006. – 320 с.

4. Zadeh, L. A. Fuzzy sets / L. A. Zadeh // Information and Control. – 1965. – Vol. 8, № 3. – P. 338–353.

5. Леоненков, А. В. Нечеткое моделирование в среде MATLAB и fuzzyTECH / А. В. Леоненков. – СПб. : БХВ-Петербург, 2005. – 736 с.

6. Python 3 Documentation [Электронный ресурс] / Python Software Foundation. – Режим доступа: https://docs.python.org/3/ (дата обращения: 04.11.2026).

7. NumPy Documentation [Электронный ресурс] / NumPy Developers. – Режим доступа: https://numpy.org/doc/stable/ (дата обращения: 04.11.2026).

8. Dantzig, G. B. The truck dispatching problem / G. B. Dantzig, J. H. Ramser // Management Science. – 1959. – Vol. 6, № 1. – P. 80–91.

9. Clarke, G. Scheduling of vehicles from a central depot to a number of delivery points / G. Clarke, J. W. Wright // Operations Research. – 1964. – Vol. 12, № 4. – P. 568–581.

10. The Vehicle Routing Problem / ed. by P. Toth, D. Vigo. – Philadelphia : SIAM, 2002. – 367 p.

11. Solomon, M. M. Algorithms for the vehicle routing and scheduling problems with time window constraints / M. M. Solomon // Operations Research. – 1987. – Vol. 35, № 2. – P. 254–265.

12. Prins, C. A simple and effective evolutionary algorithm for the vehicle routing problem / C. Prins // Computers & Operations Research. – 2004. – Vol. 31, № 12. – P. 1985–2002.

13. Джонс, М. Т. Программирование искусственного интеллекта в приложениях / М. Т. Джонс ; пер. с англ. А. И. Осипова. – М. : ДМК Пресс, 2011. – 312 с.

14. Рассел, С. Искусственный интеллект: современный подход / С. Рассел, П. Норвиг ; пер. с англ. – 2-е изд. – М. : Вильямс, 2006. – 1408 с.

15. Боровская, Е. В. Основы искусственного интеллекта [Электронный ресурс] / Е. В. Боровская, Н. А. Давыдова. – 3-е изд. (эл.). – М. : Лаборатория знаний, 2016. – 130 с. – Режим доступа: http://znanium.com/catalog/product/366789 (дата обращения: 04.11.2026).

16. Жданов, А. А. Автономный искусственный интеллект [Электронный ресурс] / А. А. Жданов. – М. : БИНОМ. Лаборатория знаний, 2012. – Режим доступа: http://znanium.com/bookread2.php?book=539035 (дата обращения: 04.11.2026).

17. Чубукова, И. А. Data Mining / И. А. Чубукова. – М. : Интернет-университет информационных технологий : БИНОМ. Лаборатория знаний, 2006. – 382 с.

18. Станкевич, Л. А. Искусственный интеллект и искусственный разум в робототехнике : учеб. пособие / Л. А. Станкевич, Е. И. Юревич. – СПб. : Изд-во Политехн. ун-та, 2012. – 167 с.

19. Штовба, С. Д. Проектирование нечетких систем средствами MATLAB / С. Д. Штовба. – М. : Горячая линия – Телеком, 2007. – 288 с.

20. Рутковская, Д. Нейронные сети, генетические алгоритмы и нечеткие системы / Д. Рутковская, М. Пилиньский, Л. Рутковский ; пер. с польск. И. Д. Рудинского. – М. : Горячая линия – Телеком, 2006. – 452 с.

21. Пегат, А. Нечеткое моделирование и управление / А. Пегат ; пер. с англ. – М. : БИНОМ. Лаборатория знаний, 2009. – 798 с.

22. Holland, J. H. Adaptation in Natural and Artificial Systems / J. H. Holland. – Ann Arbor : University of Michigan Press, 1975. – 183 p.

23. Davis, L. Applying adaptive algorithms to epistatic domains / L. Davis // Proceedings of the 9th International Joint Conference on Artificial Intelligence. – Los Angeles, 1985. – Vol. 1. – P. 162–164.

24. Mamdani, E. H. An experiment in linguistic synthesis with a fuzzy logic controller / E. H. Mamdani, S. Assilian // International Journal of Man-Machine Studies. – 1975. – Vol. 7, № 1. – P. 1–13.

# ПРИЛОЖЕНИЕ А

(обязательное)

## Листинг основных модулей программы

Ниже приведены исходные тексты модулей, реализующих интеллектуальное ядро системы. Полный исходный код, включая графический интерфейс, визуализацию и тесты, находится в каталогах src и tests электронного приложения к работе.

Листинг А.1 – Модуль src/model.py – модель задачи и показатели плана

```
"""Модель задачи маршрутизации транспорта с временными окнами (CVRPTW).

Содержит классы предметной области (клиент, транспортное средство, задача,
решение), расчёт матрицы расстояний, моделирование движения машины по
маршруту и вычисление метрик решения, которые затем подаются на вход
нечёткого блока оценки качества.
"""
from __future__ import annotations

import json
import math
from dataclasses import dataclass, field, asdict
from pathlib import Path

import numpy as np


@dataclass
class Client:
    """Клиент (точка доставки).

    Атрибуты:
        id: номер клиента (1..N; 0 зарезервирован за складом).
        x, y: координаты на плоскости, км.
        demand: объём заказа, условные единицы груза.
        urgency: срочность заказа: 1 — обычный, 2 — повышенный, 3 — срочный.
        tw_start, tw_end: временное окно обслуживания, мин от начала смены.
        service: длительность обслуживания (разгрузки), мин.
    """
    id: int
    x: float
    y: float
    demand: int
    urgency: int
    tw_start: float
    tw_end: float
    service: float = 10.0


@dataclass
class Vehicle:
    """Транспортное средство (все машины парка одинаковы)."""
    id: int
    capacity: int


@dataclass
class Instance:
    """Экземпляр задачи: склад, клиенты, парк машин и параметры смены.
```

```
    Атрибуты:
        depot: координаты склада (x, y).
        clients: список клиентов; clients[i-1] имеет id = i.
        n_vehicles: число машин K.
        capacity: грузоподъёмность одной машины.
        speed: скорость движения, км/мин (постоянная).
        horizon: длительность смены, мин (склад закрывается).
        name: имя задачи (для файлов и отчётов).
    """
    depot: tuple[float, float]
    clients: list[Client]
    n_vehicles: int
    capacity: int
    speed: float = 0.5
    horizon: float = 480.0
    name: str = "instance"
    _dist: np.ndarray | None = field(default=None, repr=False, compare=False)

    @property
    def n_clients(self) -> int:
        return len(self.clients)

    @property
    def vehicles(self) -> list[Vehicle]:
        return [Vehicle(k, self.capacity) for k in range(self.n_vehicles)]

    @property
    def total_demand(self) -> int:
        return sum(c.demand for c in self.clients)

    def coords(self) -> np.ndarray:
        """Массив координат: строка 0 — склад, строка i — клиент i."""
        pts = [self.depot] + [(c.x, c.y) for c in self.clients]
        return np.asarray(pts, dtype=float)

    @property
    def dist(self) -> np.ndarray:
        """Матрица евклидовых расстояний (N+1)×(N+1), вычисляется один раз."""
        if self._dist is None:
            p = self.coords()
            diff = p[:, None, :] - p[None, :, :]
            self._dist = np.sqrt((diff ** 2).sum(axis=2))
        return self._dist

    # ---------- сериализация в JSON (для воспроизводимости экспериментов)
----------
    def to_dict(self) -> dict:
        return {
            "name": self.name,
            "depot": list(self.depot),
            "n_vehicles": self.n_vehicles,
            "capacity": self.capacity,
            "speed": self.speed,
            "horizon": self.horizon,
            "clients": [asdict(c) for c in self.clients],
        }

    @classmethod
    def from_dict(cls, d: dict) -> "Instance":
        return cls(
            depot=tuple(d["depot"]),
            clients=[Client(**c) for c in d["clients"]],
            n_vehicles=d["n_vehicles"],
            capacity=d["capacity"],
            speed=d.get("speed", 0.5),
            horizon=d.get("horizon", 480.0),
            name=d.get("name", "instance"),
        )

    def save(self, path: str | Path) -> None:
```

```
        Path(path).write_text(json.dumps(self.to_dict(), ensure_ascii=False,
indent=2),
                              encoding="utf-8")

    @classmethod
    def load(cls, path: str | Path) -> "Instance":
        return cls.from_dict(json.loads(Path(path).read_text(encoding="utf-8")))


@dataclass
class RouteStats:
    """Показатели одного маршрута (одной машины)."""
    clients: list[int]
    distance: float
    load: int
    duration: float          # время возврата на склад, мин
    lateness: float          # суммарное опоздание, мин
    n_late: int              # число клиентов, обслуженных с опозданием
    arrivals: list[float]    # время начала обслуживания каждого клиента


@dataclass
class SolutionMetrics:
    """Сводные метрики решения — «сырьё» для нечёткой оценки качества."""
    total_distance: float
    total_lateness: float
    n_late: int
    overload: float          # суммарное превышение вместимости по всем машинам
    max_load_ratio: float    # максимальная загрузка машины, доля вместимости
    lateness_index: float    # средняя относительная величина опоздания [0..1]
    urgency_index: float     # средневзвешенное нормированное время прибытия к
срочным
    balance_cv: float        # коэффициент вариации длительностей маршрутов
    n_used: int              # число задействованных машин
    routes: list[RouteStats]


# Эталонная величина опоздания для нормировки: опоздание на 60 мин и более
# считается «полным» (вклад клиента в индекс опозданий = 1).
LATENESS_REF = 60.0


@dataclass
class Solution:
    """Решение задачи: список маршрутов, каждый — последовательность id клиентов.

    Склад (0) в маршрутах не хранится: каждый маршрут начинается и
    заканчивается на складе неявно.
    """
    routes: list[list[int]]

    def evaluate(self, inst: Instance) -> SolutionMetrics:
        return evaluate_routes(self.routes, inst)

    def to_dict(self) -> dict:
        return {"routes": self.routes}


def simulate_route(route: list[int], inst: Instance) -> RouteStats:
    """Моделирует движение машины по маршруту склад → клиенты → склад.

    Если машина приезжает раньше начала окна, она ждёт; если позже конца
    окна — фиксируется опоздание (окна мягкие).
    """
    d = inst.dist
    t = 0.0
    prev = 0
    dist = 0.0
    load = 0
    late = 0.0
```

```
    n_late = 0
    arrivals = []
    for cid in route:
        c = inst.clients[cid - 1]
        leg = float(d[prev, cid])
        dist += leg
        t += leg / inst.speed
        if t < c.tw_start:            # ожидание открытия окна
            t = c.tw_start
        arrivals.append(t)
        if t > c.tw_end:              # опоздание
            late += t - c.tw_end
            n_late += 1
        t += c.service
        load += c.demand
        prev = cid
    if route:
        leg = float(d[prev, 0])
        dist += leg
        t += leg / inst.speed
    return RouteStats(list(route), float(dist), load, t if route else 0.0, late,
n_late, arrivals)


def evaluate_routes(routes: list[list[int]], inst: Instance) -> SolutionMetrics:
    """Вычисляет все метрики решения, используемые нечёткой моделью."""
    stats = [simulate_route(r, inst) for r in routes]
    cap = inst.capacity
    total_dist = sum(s.distance for s in stats)
    total_late = sum(s.lateness for s in stats)
    n_late = sum(s.n_late for s in stats)
    overload = sum(max(0, s.load - cap) for s in stats)
    max_load = max((s.load for s in stats), default=0) / cap

    # Индекс опозданий: среднее по клиентам min(1, опоздание / 60 мин).
    lat_sum = 0.0
    # Индекс срочности: средневзвешенное (вес = urgency − 1) время начала
    # обслуживания, нормированное к длительности смены.
    urg_num = 0.0
    urg_den = 0.0
    for s in stats:
        for cid, t in zip(s.clients, s.arrivals):
            c = inst.clients[cid - 1]
            if t > c.tw_end:
                lat_sum += min(1.0, (t - c.tw_end) / LATENESS_REF)
            w = c.urgency - 1
            if w > 0:
                urg_num += w * min(1.0, t / inst.horizon)
                urg_den += w
    n = max(1, inst.n_clients)
    lateness_index = lat_sum / n
    urgency_index = urg_num / urg_den if urg_den > 0 else 0.0

    # Сбалансированность: коэффициент вариации длительностей маршрутов
    # (по всем K машинам, неиспользуемая машина имеет длительность 0).
    durs = np.array([s.duration for s in stats], dtype=float)
    if len(durs) < inst.n_vehicles:
        durs = np.concatenate([durs, np.zeros(inst.n_vehicles - len(durs))])
    mean = durs.mean() if len(durs) else 0.0
    cv = float(durs.std() / mean) if mean > 0 else 0.0

    return SolutionMetrics(
        total_distance=float(total_dist),
        total_lateness=float(total_late),
        n_late=n_late,
        overload=float(overload),
        max_load_ratio=float(max_load),
        lateness_index=lateness_index,
        urgency_index=urgency_index,
        balance_cv=cv,
```

```
        n_used=sum(1 for r in routes if r),
        routes=stats,
    )


def solution_space_size(n_clients: int, n_vehicles: int) -> float:
    """Число различных хромосом (перестановка N клиентов и K−1 одинаковых
    разделителей): (N+K−1)! / (K−1)!. Возвращается как float из-за роста."""
    return math.factorial(n_clients + n_vehicles - 1) / math.factorial(n_vehicles -
1)
```

Листинг А.2 – Модуль src/fuzzy/membership.py – функции принадлежности

```
"""Функции принадлежности нечётких множеств.

Реализованы треугольная и трапециевидная функции — самые распространённые
в нечётких системах: они задаются несколькими числами, легко
интерпретируются экспертом и быстро вычисляются.
Обе функции принимают как число, так и массив numpy.
"""
from __future__ import annotations

import numpy as np


class TrapezoidalMF:
    """Трапециевидная функция принадлежности с параметрами a ≤ b ≤ c ≤ d.

            0,               x ≤ a или x ≥ d
            (x − a)/(b − a), a < x < b      (левый склон)
    μ(x) =  1,               b ≤ x ≤ c      (ядро)
            (d − x)/(d − c), c < x < d      (правый склон)

    Если a = b (или c = d), левый (правый) склон вертикальный — так задаются
    «плечевые» термы на границах универсума (например, «короткая»).
    """

    def __init__(self, a: float, b: float, c: float, d: float):
        if not a <= b <= c <= d:
            raise ValueError(f"Нарушен порядок параметров: {a}, {b}, {c}, {d}")
        self.a, self.b, self.c, self.d = float(a), float(b), float(c), float(d)

    @property
    def params(self) -> tuple[float, ...]:
        return (self.a, self.b, self.c, self.d)

    def __call__(self, x):
        a, b, c, d = self.a, self.b, self.c, self.d
        if isinstance(x, (int, float)):           # быстрый путь для одного числа
            x = float(x)
            if x < a or x > d:
                return 0.0
            if x < b:
                return (x - a) / (b - a)
            if x <= c:
                return 1.0
            return (d - x) / (d - c)
        x = np.asarray(x, dtype=float)
        # левый склон (линейный рост); при a == b склон вертикальный
        left = (x - a) / (b - a) if b > a else np.where(x >= a, 1.0, 0.0)
        # правый склон (линейный спад); при c == d склон вертикальный
        right = (d - x) / (d - c) if d > c else np.where(x <= d, 1.0, 0.0)
        # в ядре оба склона ≥ 1, вне носителя один из них < 0 — обрезаем до [0, 1]
        mu = np.clip(np.minimum(left, right), 0.0, 1.0)
        return float(mu) if mu.ndim == 0 else mu

    def __repr__(self) -> str:
        return f"Trap({self.a:g}, {self.b:g}, {self.c:g}, {self.d:g})"
```

```
class TriangularMF(TrapezoidalMF):
    """Треугольная функция принадлежности с параметрами a ≤ b ≤ c.

    Частный случай трапеции с вырожденным ядром (b = c):
    μ(x) = max(0, min((x − a)/(b − a), (c − x)/(c − b))).
    """

    def __init__(self, a: float, b: float, c: float):
        super().__init__(a, b, b, c)

    @property
    def params(self) -> tuple[float, ...]:
        return (self.a, self.b, self.d)

    def __repr__(self) -> str:
        return f"Tri({self.a:g}, {self.b:g}, {self.d:g})"
```

Листинг А.3 – Модуль src/fuzzy/variables.py – лингвистические переменные

```
"""Лингвистические переменные нечёткой модели оценки качества маршрутов.

Лингвистическая переменная — это переменная, значениями которой являются
слова (термы), например «короткая», «средняя», «длинная». Каждый терм —
нечёткое множество на универсуме (числовой области определения) с
собственной функцией принадлежности.

Пять входных переменных описывают решение задачи маршрутизации целиком,
одна выходная — его итоговое качество.
"""
from __future__ import annotations

from dataclasses import dataclass, field

from src.fuzzy.membership import TrapezoidalMF, TriangularMF


@dataclass
class Term:
    """Терм лингвистической переменной: имя, подпись и функция принадлежности."""
    name: str
    label: str
    mf: TrapezoidalMF


@dataclass
class LinguisticVariable:
    """Лингвистическая переменная.

    Атрибуты:
        name: идентификатор в программе (латиница).
        title: название для человека («Протяжённость»).
        universe: границы универсума (min, max).
        terms: термы переменной в порядке «от хорошего к плохому» или по
возрастанию.
        description: как вычисляется чёткое значение.
    """
    name: str
    title: str
    universe: tuple[float, float]
    terms: dict[str, Term] = field(default_factory=dict)
    description: str = ""

    def add(self, name: str, label: str, mf: TrapezoidalMF) -> "LinguisticVariable":
        self.terms[name] = Term(name, label, mf)
        return self
```

```
    def clip(self, x: float) -> float:
        """Приведение чёткого значения к границам универсума."""
        lo, hi = self.universe
        return min(max(x, lo), hi)

    def fuzzify(self, x: float) -> dict[str, float]:
        """Фаззификация: степени принадлежности x ко всем термам переменной."""
        x = self.clip(x)
        return {t.name: float(t.mf(x)) for t in self.terms.values()}


def make_input_variables() -> dict[str, LinguisticVariable]:
    """Создаёт пять входных лингвистических переменных.

    Параметры функций принадлежности подобраны по диапазонам значений,
    наблюдаемым на сгенерированных задачах (от случайных решений до
    решений без опозданий). Соседние термы образуют разбиение Руспини:
    склоны соседних термов пересекаются на уровне 0,5, сумма степеней
    принадлежности равна 1. Поэтому почти любое значение принадлежит
    сразу двум термам, и оценка качества меняется плавно — у ГА нет
    «плато», на которых приспособленность не реагирует на улучшения.
    """
    length = LinguisticVariable(
        "length", "Протяжённость", (0.6, 2.6),
        description="суммарный пробег, делённый на пробег жадного решения "
                    "«ближайший сосед» (1,0 — как у жадного алгоритма)")
    length.add("short", "короткая", TrapezoidalMF(0.6, 0.6, 0.8, 1.1))
    length.add("medium", "средняя", TriangularMF(0.8, 1.1, 1.6))
    length.add("long", "длинная", TrapezoidalMF(1.1, 1.6, 2.6, 2.6))

    load = LinguisticVariable(
        "load", "Загрузка транспорта", (0.0, 2.0),
        description="максимальная загрузка машины в долях от вместимости")
    load.add("low", "низкая", TrapezoidalMF(0.0, 0.0, 0.5, 0.8))
    load.add("normal", "нормальная", TrapezoidalMF(0.5, 0.8, 1.0, 1.1))
    load.add("over", "перегрузка", TrapezoidalMF(1.0, 1.1, 2.0, 2.0))

    tw = LinguisticVariable(
        "timewin", "Соблюдение временных окон", (0.0, 1.0),
        description="средняя по клиентам величина min(1, опоздание / 60 мин)")
    tw.add("good", "хорошее", TriangularMF(0.0, 0.0, 0.12))
    tw.add("fair", "удовлетворительное", TriangularMF(0.0, 0.12, 0.45))
    tw.add("poor", "плохое", TrapezoidalMF(0.12, 0.45, 1.0, 1.0))

    urg = LinguisticVariable(
        "urgency", "Обслуживание срочных заказов", (0.0, 1.0),
        description="средневзвешенное по срочности время начала обслуживания "
                    "срочных клиентов, делённое на длительность смены")
    urg.add("good", "хорошее", TrapezoidalMF(0.0, 0.0, 0.05, 0.25))
    urg.add("medium", "среднее", TriangularMF(0.05, 0.25, 0.5))
    urg.add("poor", "плохое", TrapezoidalMF(0.25, 0.5, 1.0, 1.0))

    bal = LinguisticVariable(
        "balance", "Сбалансированность", (0.0, 1.5),
        description="коэффициент вариации длительностей маршрутов машин")
    bal.add("even", "равномерная", TrapezoidalMF(0.0, 0.0, 0.05, 0.4))
    bal.add("uneven", "неравномерная", TrapezoidalMF(0.05, 0.4, 1.5, 1.5))

    return {v.name: v for v in (length, load, tw, urg, bal)}


def make_output_variable() -> LinguisticVariable:
    """Выходная переменная «Качество решения» на универсуме [0, 1]."""
    q = LinguisticVariable("quality", "Качество решения", (0.0, 1.0),
                           description="интегральная оценка плана маршрутов")
    q.add("very_low", "очень низкое", TriangularMF(0.0, 0.0, 0.25))
    q.add("low", "низкое", TriangularMF(0.0, 0.25, 0.5))
    q.add("medium", "среднее", TriangularMF(0.25, 0.5, 0.75))
    q.add("high", "высокое", TriangularMF(0.5, 0.75, 1.0))
    q.add("very_high", "очень высокое", TriangularMF(0.75, 1.0, 1.0))
```

```
    return q
```

Листинг А.4 – Модуль src/fuzzy/rules.py – база нечётких правил

```
"""База нечётких продукционных правил «ЕСЛИ – ТО» с весовыми коэффициентами.

Формат правила (по методическим указаниям, раздел 3.3.2):
    RULE_j: ЕСЛИ «условие_1» И … И «условие_m» ТО «заключение» (F_j),
где F_j Є (0, 1] — весовой коэффициент (степень уверенности эксперта
в истинности заключения). Каждое правило снабжено пояснением.

Правила 1–9 перебирают все сочетания термов «Протяжённости» и «Соблюдения
временных окон». Так как термы каждой переменной покрывают весь универсум,
при любых входных данных срабатывает хотя бы одно из них (полнота базы
правил), и дефаззификация всегда определена.
"""
from __future__ import annotations

from dataclasses import dataclass


@dataclass(frozen=True)
class Rule:
    """Нечёткое продукционное правило.

    Атрибуты:
        conditions: подусловия (имя_переменной, имя_терма), связанные «И».
        conclusion: имя терма выходной переменной «Качество решения».
        weight: весовой коэффициент F.
        explanation: пояснение эксперта.
    """
    conditions: tuple[tuple[str, str], ...]
    conclusion: str
    weight: float
    explanation: str


def R(cond: dict, concl: str, w: float, expl: str) -> Rule:
    return Rule(tuple(cond.items()), concl, w, expl)


RULES: list[Rule] = [
    # --- 1–9. Ядро базы: таблица «Протяжённость × Соблюдение окон» ---
    # Таблица монотонна: улучшение любого из двух показателей на один терм
    # повышает заключение на один терм (от «очень низкого» до «очень высокого»).
    R({"length": "short", "timewin": "good"}, "very_high", 1.0,
      "Короткий пробег без опозданий — главная цель планирования доставки."),
    R({"length": "short", "timewin": "fair"}, "high", 1.0,
      "Пробег экономичен, опоздания единичны и невелики."),
    R({"length": "short", "timewin": "poor"}, "medium", 1.0,
      "Экономия пробега получена ценой массовых опозданий — компромиссный план."),
    R({"length": "medium", "timewin": "good"}, "high", 1.0,
      "Опозданий нет, перерасход пробега умеренный."),
    R({"length": "medium", "timewin": "fair"}, "medium", 1.0,
      "Средний пробег и отдельные опоздания — план посредственный."),
    R({"length": "medium", "timewin": "poor"}, "low", 1.0,
      "Пробег не сокращён, а клиенты обслуживаются с опозданием."),
    R({"length": "long", "timewin": "good"}, "medium", 1.0,
      "Окна соблюдены, но за счёт большого перерасхода топлива и времени."),
    R({"length": "long", "timewin": "fair"}, "low", 1.0,
      "Длинные маршруты не спасают от опозданий — план неэффективен."),
    R({"length": "long", "timewin": "poor"}, "very_low", 1.0,
      "Большой пробег и массовые опоздания одновременно — план следует
отвергнуть."),
    # --- 10–15. Обслуживание срочных заказов ---
    # Положительные правила срабатывают только вместе с хорошими значениями
    # пробега или окон: хороший сервис срочных клиентов не «спасает» плохой план.
    R({"urgency": "good", "timewin": "good"}, "very_high", 0.8,
```

```
      "Срочные клиенты обслужены в начале смены, опозданий нет."),
    R({"urgency": "good", "length": "short"}, "high", 0.7,
      "Срочные заказы доставлены первыми без перерасхода пробега."),
    R({"urgency": "medium", "timewin": "good"}, "high", 0.6,
      "Опозданий нет, срочные заказы доставлены не первыми, но вовремя."),
    R({"urgency": "medium", "timewin": "fair"}, "medium", 0.6,
      "Посредственное обслуживание и срочных, и обычных клиентов."),
    R({"urgency": "poor"}, "low", 0.8,
      "Срочные заказы доставляются в конце смены — риск потери клиентов."),
    R({"urgency": "poor", "timewin": "poor"}, "very_low", 1.0,
      "Опоздания вместе с поздней доставкой срочных заказов — худший сервис."),
    # --- 16–18. Загрузка транспорта ---
    R({"load": "over"}, "very_low", 1.0,
      "Перегруз машины физически недопустим (дополнительно штрафуется вне нечёткого
блока)."),
    R({"load": "normal", "length": "short", "timewin": "good"}, "very_high", 0.8,
      "Рациональная загрузка кузова при коротком пробеге и без опозданий."),
    R({"load": "low", "length": "long"}, "low", 0.8,
      "Машины недогружены, но много ездят — парк используется неэффективно."),
    # --- 19–23. Сбалансированность нагрузки водителей ---
    R({"balance": "even", "timewin": "good"}, "high", 0.7,
      "Нагрузка водителей равномерна, сверхурочных нет, клиенты обслужены
вовремя."),
    R({"balance": "even", "length": "short"}, "high", 0.6,
      "Равные по длительности маршруты при экономичном пробеге."),
    R({"balance": "uneven"}, "low", 0.4,
      "Один водитель перегружен, другие простаивают."),
    R({"balance": "uneven", "timewin": "fair"}, "low", 0.7,
      "Перегруженный водитель опаздывает, пока другие простаивают."),
    R({"balance": "uneven", "length": "medium"}, "low", 0.6,
      "Неравномерная нагрузка при среднем пробеге — признак неудачного распределения
заказов."),
    # --- 24–26. Комбинированные правила ---
    R({"length": "short", "urgency": "good", "balance": "even"}, "very_high", 0.9,
      "Экономичный план с приоритетом срочных заказов и равной нагрузкой
водителей."),
    R({"length": "long", "urgency": "poor"}, "very_low", 0.8,
      "Длинные маршруты, на которых срочные заказы доставляются последними."),
    R({"length": "medium", "urgency": "good"}, "high", 0.6,
      "Небольшой перерасход пробега оправдан ранней доставкой срочных заказов."),
]


def rule_to_text(rule: Rule, variables: dict, output) -> str:
    """Текстовая запись правила: «ЕСЛИ Протяжённость = короткая И … ТО Качество =
…»."""
    conds = " И ".join(
        f"{variables[v].title} = «{variables[v].terms[t].label}»" for v, t in
rule.conditions)
    return f"ЕСЛИ {conds} ТО {output.title} =
«{output.terms[rule.conclusion].label}» " \
           f"(F = {rule.weight:g})"
```

Листинг А.5 – Модуль src/fuzzy/mamdani.py – механизм вывода Мамдани

```
"""Механизм нечёткого логического вывода по алгоритму Мамдани.

Этапы (по методическим указаниям, раздел 3.3.2):
    1. Формирование базы правил       — конструктор MamdaniInference;
    2. Фаззификация входных переменных — fuzzify:     b_i = μ(a_i);
    3. Агрегирование подусловий         — aggregate:   c_j = min{b_i};
    4. Активизация подзаключений        — activate:    d_j = c_j·F_j,
                                                       μ'_j(x) = min{d_j, μ_j(x)};
    5. Аккумуляция заключений           — accumulate:  μ_E(x) = max_j μ'_j(x);
    6. Дефаззификация (центр тяжести)   — defuzzify:   y = ∫x·μ_E(x)dx / ∫μ_E(x)dx.

Универсум выходной переменной дискретизируется сеткой из ``resolution``
точек, интегралы центра тяжести заменяются суммами (метод прямоугольников).
```

```
"""
from __future__ import annotations

from dataclasses import dataclass

import numpy as np

from src.fuzzy.rules import Rule
from src.fuzzy.variables import LinguisticVariable


@dataclass
class InferenceTrace:
    """Полный протокол одного нечёткого вывода (для отчёта и GUI)."""
    inputs: dict[str, float]                        # чёткие входные значения a_i
    fuzzified: dict[str, dict[str, float]]          # b: переменная → терм → μ
    strengths: list[float]                           # c_j — степень истинности
условий
    activations: list[float]                         # d_j = c_j · F_j
    grid: np.ndarray                                 # сетка универсума выхода
    activated_sets: np.ndarray                       # μ'_j(x), матрица n_rules ×
len(grid)
    accumulated: np.ndarray                          # μ_E(x)
    output: float                                    # y — результат дефаззификации


class MamdaniInference:
    """Нечёткая система вывода Мамдани с одной выходной переменной."""

    def __init__(self, inputs: dict[str, LinguisticVariable], output:
LinguisticVariable,
                 rules: list[Rule], resolution: int = 201):
        # Этап 1. Формирование базы правил: проверяем, что все переменные и
        # термы, упомянутые в правилах, существуют.
        for j, r in enumerate(rules, 1):
            for v, t in r.conditions:
                if v not in inputs or t not in inputs[v].terms:
                    raise ValueError(f"Правило {j}: неизвестный терм {v}={t}")
            if r.conclusion not in output.terms:
                raise ValueError(f"Правило {j}: неизвестный выходной терм
{r.conclusion}")
        self.inputs = inputs
        self.output = output
        self.rules = rules
        lo, hi = output.universe
        self.grid = np.linspace(lo, hi, resolution)
        # Функции принадлежности термов-заключений, заранее вычисленные на
        # сетке: строка j — μ терма из заключения правила j.
        self._concl_mf = np.vstack([output.terms[r.conclusion].mf(self.grid) for r
in rules])
        self._weights = np.array([r.weight for r in rules])

    # ------------------------------------------------------------------ этапы
    def fuzzify(self, values: dict[str, float]) -> dict[str, dict[str, float]]:
        """Этап 2. Фаззификация: для каждой входной переменной вычисляются
        степени принадлежности её чёткого значения ко всем термам."""
        return {name: var.fuzzify(values[name]) for name, var in
self.inputs.items()}

    def aggregate(self, fuzz: dict[str, dict[str, float]]) -> np.ndarray:
        """Этап 3. Агрегирование подусловий: степень истинности условия
        правила равна минимуму степеней истинности его подусловий (нечёткое «И»)."""
        # c_j = min{b_i} по всем подусловиям (переменная v, терм t) правила j
        return np.array([min(fuzz[v][t] for v, t in r.conditions) for r in
self.rules])

    def activate(self, c: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
        """Этап 4. Активизация: d_j = c_j·F_j; функция принадлежности
        заключения «срезается» на уровне d_j (min-активизация)."""
        d = c * self._weights                                  # d_j = c_j · F_j
```

```
        # μ'_j(x) = min(d_j, μ_j(x)): строка j — «срезанный» терм заключения правила
j
        activated = np.minimum(d[:, None], self._concl_mf)
        return d, activated

    @staticmethod
    def accumulate(activated: np.ndarray) -> np.ndarray:
        """Этап 5. Аккумуляция: объединение (max) всех активизированных множеств."""
        return activated.max(axis=0)                          # μ_E(x) = max_j
μ'_j(x)

    def defuzzify(self, mu: np.ndarray) -> float:
        """Этап 6. Дефаззификация методом центра тяжести.

        Если ни одно правило не сработало (площадь равна нулю), возвращается
        0 — худшая оценка. При полной базе правил такого не происходит.
        """
        area = mu.sum()                                       # Σ μ_E(x_g)
        if area <= 1e-12:
            return 0.0
        return float((self.grid * mu).sum() / area)           # Σ x_g·μ_E(x_g) / Σ
μ_E(x_g)

    # ------------------------------------------------------------- вывод целиком
    def infer(self, values: dict[str, float]) -> float:
        """Чёткие входы → чёткая оценка качества (этапы 2–6)."""
        fuzz = self.fuzzify(values)
        c = self.aggregate(fuzz)
        _, activated = self.activate(c)
        return self.defuzzify(self.accumulate(activated))

    def trace(self, values: dict[str, float]) -> InferenceTrace:
        """То же, что infer, но с сохранением всех промежуточных результатов."""
        fuzz = self.fuzzify(values)
        c = self.aggregate(fuzz)
        d, activated = self.activate(c)
        acc = self.accumulate(activated)
        return InferenceTrace(dict(values), fuzz, c.tolist(), d.tolist(), self.grid,
                              activated, acc, self.defuzzify(acc))


def build_default_engine(resolution: int = 201) -> MamdaniInference:
    """Создаёт систему вывода с переменными и базой правил по умолчанию."""
    from src.fuzzy.rules import RULES
    from src.fuzzy.variables import make_input_variables, make_output_variable
    return MamdaniInference(make_input_variables(), make_output_variable(), RULES,
resolution)
```

Листинг А.6 – Модуль src/ga/encoding.py – кодирование хромосомы

```
"""Кодирование решения в хромосому и обратно.

Хромосома — перестановка длины N+K−1 из чисел 1..N+K−1:
    * гены 1..N — номера клиентов (порядок объезда);
    * гены N+1..N+K−1 — разделители между маршрутами машин.

Пример (N=6, K=3, разделители 7 и 8):
    [3, 1, 7, 6, 2, 8, 5, 4]  →  машина 1: 3→1, машина 2: 6→2, машина 3: 5→4

Так одна хромосома одновременно задаёт и распределение заказов по машинам,
и порядок объезда внутри каждого маршрута.
"""
from __future__ import annotations

import numpy as np


def is_separator(gene: int, n_clients: int) -> bool:
```

```
    return gene > n_clients


def decode(chrom, n_clients: int) -> list[list[int]]:
    """Хромосома → список маршрутов (разрезание по разделителям)."""
    routes: list[list[int]] = [[]]
    for g in chrom:
        if g > n_clients:
            routes.append([])
        else:
            routes[-1].append(int(g))
    return routes


def encode(routes: list[list[int]], n_clients: int) -> list[int]:
    """Список маршрутов → хромосома (маршруты, склеенные через разделители)."""
    chrom: list[int] = []
    sep = n_clients + 1
    for i, r in enumerate(routes):
        if i > 0:
            chrom.append(sep)
            sep += 1
        chrom.extend(r)
    return chrom


def random_chromosome(n_clients: int, n_vehicles: int, rng: np.random.Generator) ->
list[int]:
    """Случайная хромосома: случайная перестановка клиентов и разделителей."""
    genes = np.arange(1, n_clients + n_vehicles)
    rng.shuffle(genes)
    return genes.tolist()


def is_valid(chrom, n_clients: int, n_vehicles: int) -> bool:
    """Проверка, что хромосома — перестановка 1..N+K−1 (каждый ген ровно один
раз)."""
    return sorted(chrom) == list(range(1, n_clients + n_vehicles))
```

Листинг А.7 – Модуль src/ga/operators.py – генетические операторы

```
"""Генетические операторы: отбор, скрещивание, мутации.

Все операторы работают с хромосомой-перестановкой (см. encoding.py) и
сохраняют её корректность: каждый клиент и каждый разделитель после
операции встречается ровно один раз.
"""
from __future__ import annotations

import numpy as np


# ============================================================================ ОТБОР
def tournament_selection(fits: np.ndarray, rng: np.random.Generator, k: int = 3) ->
int:
    """Турнирный отбор: из k случайных особей выбирается лучшая.

    Давление отбора регулируется размером турнира k; не зависит от масштаба
    значений приспособленности (важно, т.к. штраф делает их отрицательными).
    """
    idx = rng.integers(0, len(fits), size=k)
    return int(idx[np.argmax(fits[idx])])


def roulette_selection(fits: np.ndarray, rng: np.random.Generator, **_) -> int:
    """Пропорциональный (рулеточный) отбор.

    Вероятность выбора особи пропорциональна её приспособленности,
```

```
    предварительно сдвинутой так, чтобы минимум стал малым положительным
    числом (приспособленность со штрафом может быть отрицательной).
    """
    shifted = fits - fits.min() + 1e-6
    p = shifted / shifted.sum()
    return int(rng.choice(len(fits), p=p))


def rank_selection(fits: np.ndarray, rng: np.random.Generator, **_) -> int:
    """Ранговый отбор: вероятность пропорциональна рангу особи (1 — худшая,
    P — лучшая). Устраняет доминирование «сверхособей» на ранних поколениях."""
    ranks = np.empty(len(fits))
    ranks[np.argsort(fits)] = np.arange(1, len(fits) + 1)
    return int(rng.choice(len(fits), p=ranks / ranks.sum()))


SELECTION = {
    "tournament": tournament_selection,
    "roulette": roulette_selection,
    "rank": rank_selection,
}


# ================================================================== СКРЕЩИВАНИЕ
def order_crossover(p1: list[int], p2: list[int], rng: np.random.Generator) ->
list[int]:
    """Упорядоченное скрещивание (Order Crossover, OX).

    1. Случайно выбирается отрезок [i, j) и копируется из первого родителя
       в потомка на те же позиции.
    2. Оставшиеся позиции, начиная с j и по кругу, заполняются генами второго
       родителя в порядке их следования (также начиная с позиции j),
       пропуская уже скопированные гены.
    Потомок наследует от первого родителя фрагмент маршрута, а от второго —
    относительный порядок остальных клиентов. Результат — всегда перестановка.
    """
    n = len(p1)
    i, j = sorted(rng.choice(n + 1, size=2, replace=False))
    child = [None] * n
    child[i:j] = p1[i:j]
    taken = set(p1[i:j])
    fill = [p2[(j + k) % n] for k in range(n) if p2[(j + k) % n] not in taken]
    pos = j % n
    for g in fill:
        while child[pos] is not None:
            pos = (pos + 1) % n
        child[pos] = g
        pos = (pos + 1) % n
    return child


# ================================================================== МУТАЦИИ
def swap_mutation(chrom: list[int], rng: np.random.Generator, n_clients: int) ->
list[int]:
    """Обмен двух случайных генов (в т.ч. клиентов разных маршрутов)."""
    c = list(chrom)
    i, j = rng.choice(len(c), size=2, replace=False)
    c[i], c[j] = c[j], c[i]
    return c


def inversion_mutation(chrom: list[int], rng: np.random.Generator, n_clients: int) -
> list[int]:
    """Инверсия случайного отрезка: порядок генов на нём меняется на обратный.

    Если отрезок лежит внутри одного маршрута, это аналог хода 2-opt —
    устраняет самопересечения маршрута.
    """
    c = list(chrom)
    i, j = sorted(rng.choice(len(c) + 1, size=2, replace=False))
```

```
    c[i:j] = c[i:j][::-1]
    return c


def transfer_mutation(chrom: list[int], rng: np.random.Generator, n_clients: int) ->
list[int]:
    """Перенос клиента в другой маршрут: случайный клиент удаляется из своего
    маршрута и вставляется в случайную позицию другого маршрута.

    Меняет распределение заказов по машинам, не затрагивая порядок
    остальных клиентов (важно для выравнивания загрузки).
    """
    c = list(chrom)
    client_pos = [p for p, g in enumerate(c) if g <= n_clients]
    p = int(rng.choice(client_pos))
    gene = c.pop(p)
    # номера маршрутов: маршрут r — между (r−1)-м и r-м разделителем
    seps = [q for q, g in enumerate(c) if g > n_clients]
    bounds = [-1] + seps + [len(c)]
    own = sum(1 for q in seps if q < p)         # маршрут, из которого удалён клиент
    others = [r for r in range(len(bounds) - 1) if r != own]
    if not others:                               # единственная машина
        c.insert(p, gene)
        return c
    r = int(rng.choice(others))
    lo, hi = bounds[r] + 1, bounds[r + 1]        # допустимые позиции вставки [lo,
hi]
    c.insert(int(rng.integers(lo, hi + 1)), gene)
    return c


MUTATIONS = (swap_mutation, inversion_mutation, transfer_mutation)


def mutate(chrom: list[int], rng: np.random.Generator, n_clients: int) -> list[int]:
    """Применяет один из трёх операторов мутации, выбранный равновероятно."""
    op = MUTATIONS[int(rng.integers(len(MUTATIONS)))]
    return op(chrom, rng, n_clients)
```

Листинг А.8 – Модуль src/ga/fitness.py – функции приспособленности

```
"""Функции приспособленности для генетического алгоритма.

Главная — ``FuzzyFitness``: здесь и происходит гибридизация методов ИИ.
Для каждой особи каждого поколения ГА вызывает нечёткий вывод Мамдани,
и полученная оценка качества Q ∈ [0, 1] становится приспособленностью.

Для сравнения реализована ``WeightedSumFitness`` — та же схема, но вместо
нечёткого блока используется линейная взвешенная сумма тех же показателей.
"""
from __future__ import annotations

from src.baselines import nearest_neighbor
from src.fuzzy.mamdani import MamdaniInference, build_default_engine
from src.model import Instance, SolutionMetrics, evaluate_routes

# Штраф за перегруз: любое решение с перегрузом получает приспособленность
# не выше Q_max − 1 < 0, т.е. хуже любого допустимого решения (Q ≥ 0);
# слагаемое, пропорциональное величине перегруза, сохраняет «градиент»
# среди недопустимых решений, чтобы ГА мог выбираться из этой области.
OVERLOAD_BASE = 1.0
OVERLOAD_RATE = 1.0


def overload_penalty(m: SolutionMetrics, capacity: int) -> float:
    """Штраф за нарушение жёсткого ограничения вместимости."""
    if m.overload <= 0:
        return 0.0
```

```
    return OVERLOAD_BASE + OVERLOAD_RATE * m.overload / capacity


def fuzzy_inputs(m: SolutionMetrics, ref_distance: float) -> dict[str, float]:
    """Чёткие значения пяти входных лингвистических переменных."""
    return {
        "length": m.total_distance / ref_distance,
        "load": m.max_load_ratio,
        "timewin": m.lateness_index,
        "urgency": m.urgency_index,
        "balance": m.balance_cv,
    }


class FuzzyFitness:
    """Приспособленность = нечёткая оценка качества − штраф за перегруз.

    Эталонный пробег для нормировки «Протяжённости» — пробег жадного
    решения «ближайший сосед» на той же задаче (вычисляется один раз).
    Значения кэшируются: одинаковые особи (частые после элитизма и
    скрещивания похожих родителей) повторно не оцениваются.
    """

    name = "fuzzy"

    def __init__(self, inst: Instance, engine: MamdaniInference | None = None):
        self.inst = inst
        self.engine = engine or build_default_engine()
        self.ref_distance = evaluate_routes(nearest_neighbor(inst),
inst).total_distance
        self.cache: dict[tuple, float] = {}
        self.evaluations = 0

    def quality(self, m: SolutionMetrics) -> float:
        """Нечёткая оценка Q (без штрафа) — вызов вывода Мамдани."""
        return self.engine.infer(fuzzy_inputs(m, self.ref_distance))

    def score(self, routes: list[list[int]]) -> float:
        """Приспособленность решения, заданного списком маршрутов."""
        m = evaluate_routes(routes, self.inst)
        return self.quality(m) - overload_penalty(m, self.inst.capacity)

    def __call__(self, chrom) -> float:
        from src.ga.encoding import decode
        key = tuple(chrom)
        val = self.cache.get(key)
        if val is None:                    # особь ещё не оценивалась
            self.evaluations += 1
            # хромосома → маршруты → метрики → нечёткий вывод Мамдани − штраф
            val = self.score(decode(chrom, self.inst.n_clients))
            self.cache[key] = val
        return val


class WeightedSumFitness(FuzzyFitness):
    """Альтернатива для сравнения: линейная свёртка тех же показателей.

    Q_ws = 1 − (w_L·s_L + w_T·s_T + w_U·s_U + w_B·s_B), где s — показатели,
    нормированные к [0, 1] по границам универсумов нечётких переменных.
    Перегруз штрафуется так же, как и в нечёткой схеме.
    """

    name = "weighted"
    WEIGHTS = {"length": 0.35, "timewin": 0.30, "urgency": 0.20, "balance": 0.15}

    def quality(self, m: SolutionMetrics) -> float:
        x = fuzzy_inputs(m, self.ref_distance)
        s = {}
        for k in self.WEIGHTS:
            lo, hi = self.engine.inputs[k].universe
```

```
            s[k] = (min(max(x[k], lo), hi) - lo) / (hi - lo)
        return 1.0 - sum(w * s[k] for k, w in self.WEIGHTS.items())
```

Листинг А.9 – Модуль src/ga/engine.py – цикл генетического алгоритма

```
"""Главный цикл генетического алгоритма.

Схема одного поколения:
    1. оценка приспособленности всех особей (нечёткий вывод Мамдани);
    2. элитизм — лучшие ``elite`` особей переходят в новое поколение без изменений;
    3. пока новое поколение не заполнено: отбор двух родителей →
       скрещивание OX с вероятностью pc → мутация каждого потомка с вероятностью pm;
    4. проверка критериев останова: лимит поколений, стагнация, лимит времени.
"""
from __future__ import annotations

import time
from dataclasses import dataclass, field
from typing import Callable

import numpy as np

from src.baselines import randomized_nearest_neighbor
from src.ga.encoding import decode, encode, random_chromosome
from src.ga.operators import SELECTION, mutate, order_crossover
from src.model import Instance


@dataclass
class GAConfig:
    """Параметры генетического алгоритма (значения по умолчанию обоснованы
    экспериментами раздела 4 пояснительной записки)."""
    pop_size: int = 100
    generations: int = 300
    crossover_rate: float = 0.9
    mutation_rate: float = 0.2
    selection: str = "tournament"
    tournament_k: int = 3
    elite: int = 2
    seed_fraction: float = 0.1     # доля начальной популяции из рандомизированной
эвристики
    stagnation: int = 120          # останов, если лучшая особь не улучшалась
столько поколений
    time_limit: float | None = None
    seed: int = 0


@dataclass
class GAResult:
    """Результат работы ГА и журнал сходимости."""
    best_chrom: list[int]
    best_routes: list[list[int]]
    best_fitness: float
    history_best: list[float] = field(default_factory=list)
    history_mean: list[float] = field(default_factory=list)
    history_worst: list[float] = field(default_factory=list)
    initial_best_routes: list[list[int]] | None = None
    generations_run: int = 0
    evaluations: int = 0
    elapsed: float = 0.0
    stop_reason: str = ""


def run_ga(inst: Instance, fitness: Callable[[list[int]], float], cfg: GAConfig |
None = None,
           callback: Callable[[int, float, float, list[int]], bool | None] | None =
None
           ) -> GAResult:
```

```
    """Запускает ГА на задаче ``inst`` с функцией приспособленности ``fitness``.

    callback(поколение, лучшая, средняя, лучшая_хромосома) вызывается после
    каждого поколения (используется GUI); если он вернёт True — останов.
    """
    cfg = cfg or GAConfig()
    rng = np.random.default_rng(cfg.seed)
    n, k = inst.n_clients, inst.n_vehicles
    select = SELECTION[cfg.selection]
    t0 = time.perf_counter()

    # Начальная популяция: небольшая доля (seed_fraction) — решения
    # рандомизированного «ближайшего соседа», остальные — случайные
    # перестановки. «Засев» выводит часть особей в область, где нечёткая
    # оценка информативна, а случайные особи обеспечивают разнообразие.
    n_seed = int(round(cfg.seed_fraction * cfg.pop_size))
    pop = [encode(randomized_nearest_neighbor(inst, rng), n) for _ in range(n_seed)]
    pop += [random_chromosome(n, k, rng) for _ in range(cfg.pop_size - n_seed)]
    fits = np.array([fitness(c) for c in pop])

    best_i = int(np.argmax(fits))
    best_chrom, best_fit = list(pop[best_i]), float(fits[best_i])
    res = GAResult(best_chrom, decode(best_chrom, n), best_fit,
                   initial_best_routes=decode(best_chrom, n))
    last_improve = 0
    gen = 0
    res.stop_reason = "лимит поколений"
    for gen in range(1, cfg.generations + 1):
        # --- журнал сходимости (по текущему поколению)
        res.history_best.append(float(fits.max()))
        res.history_mean.append(float(fits.mean()))
        res.history_worst.append(float(fits.min()))

        # --- элитизм: лучшие особи копируются без изменений
        order = np.argsort(fits)[::-1]
        new_pop = [list(pop[i]) for i in order[:cfg.elite]]

        # --- отбор, скрещивание, мутация
        while len(new_pop) < cfg.pop_size:
            p1 = pop[select(fits, rng, k=cfg.tournament_k)]
            p2 = pop[select(fits, rng, k=cfg.tournament_k)]
            if rng.random() < cfg.crossover_rate:
                c1, c2 = order_crossover(p1, p2, rng), order_crossover(p2, p1, rng)
            else:
                c1, c2 = list(p1), list(p2)
            for child in (c1, c2):
                if rng.random() < cfg.mutation_rate:
                    child = mutate(child, rng, n)
                if len(new_pop) < cfg.pop_size:
                    new_pop.append(child)
        pop = new_pop
        fits = np.array([fitness(c) for c in pop])

        # --- обновление лучшего решения
        i = int(np.argmax(fits))
        if fits[i] > best_fit + 1e-12:
            best_fit, best_chrom = float(fits[i]), list(pop[i])
            last_improve = gen
        if callback is not None and callback(gen, best_fit, float(fits.mean()),
best_chrom):
            res.stop_reason = "остановлено пользователем"
            break
        # --- критерии останова
        if gen - last_improve >= cfg.stagnation:
            res.stop_reason = f"стагнация {cfg.stagnation} поколений"
            break
        if cfg.time_limit is not None and time.perf_counter() - t0 > cfg.time_limit:
            res.stop_reason = "лимит времени"
            break
```

```
    res.history_best.append(float(fits.max()))
    res.history_mean.append(float(fits.mean()))
    res.history_worst.append(float(fits.min()))
    res.best_chrom, res.best_fitness = best_chrom, best_fit
    res.best_routes = decode(best_chrom, n)
    res.generations_run = gen
    res.evaluations = getattr(fitness, "evaluations", 0)
    res.elapsed = time.perf_counter() - t0
    return res
```

## ПРИЛОЖЕНИЕ Б

(справочное)

### Пример входного файла задачи

Задача с N = 10 клиентами и K = 2 машинами (файл experiments/instances/N10_s1.json). Поля клиента: id – номер, x и y – координаты, км; demand – объём заказа; urgency – срочность (1 – обычный, 2 – повышенный, 3 – срочный); tw_start и tw_end – временное окно, мин от начала смены; service – время разгрузки, мин.

```
{
  "name": "N10_s1", "depot": [20.0, 20.0], "n_vehicles": 2,
  "capacity": 32, "speed": 0.5, "horizon": 480.0,
  "clients": [
    {"id": 1, "x": 20.47, "y": 38.02, "demand": 1, "urgency": 2, "tw_start": 49.2,
"tw_end": 185.8, "service": 10.0},
    {"id": 2, "x": 5.77, "y": 37.95, "demand": 8, "urgency": 1, "tw_start": 96.1,
"tw_end": 261.2, "service": 10.0},
    {"id": 3, "x": 12.47, "y": 16.93, "demand": 1, "urgency": 1, "tw_start": 0.0,
"tw_end": 97.2, "service": 10.0},
    {"id": 4, "x": 33.11, "y": 16.37, "demand": 3, "urgency": 1, "tw_start": 157.6,
"tw_end": 359.9, "service": 10.0},
    {"id": 5, "x": 21.98, "y": 1.1, "demand": 5, "urgency": 3, "tw_start": 0.0,
"tw_end": 60.8, "service": 10.0},
    {"id": 6, "x": 30.14, "y": 21.53, "demand": 5, "urgency": 1, "tw_start": 136.8,
"tw_end": 333.0, "service": 10.0},
    {"id": 7, "x": 13.19, "y": 31.54, "demand": 2, "urgency": 1, "tw_start": 28.8,
"tw_end": 209.9, "service": 10.0},
    {"id": 8, "x": 12.13, "y": 18.14, "demand": 10, "urgency": 2, "tw_start": 0.0,
"tw_end": 69.0, "service": 10.0},
    {"id": 9, "x": 5.36, "y": 16.12, "demand": 8, "urgency": 2, "tw_start": 0.0,
"tw_end": 104.5, "service": 10.0},
    {"id": 10, "x": 8.14, "y": 10.49, "demand": 10, "urgency": 2, "tw_start": 10.0,
"tw_end": 133.6, "service": 10.0}
  ]
}
```

## ПРИЛОЖЕНИЕ В

(справочное)

### Пример протокола работы программы

Вывод консольного интерфейса при решении задачи с N = 20 клиентами (команда python -m src.main --clients 20 --seed 2 --trace): ход эволюции, сравнение с жадным алгоритмом, маршруты машин и полный протокол нечёткого вывода для найденного решения.

```
Задача N20_K3_s2: N = 20, K = 3, вместимость 38, суммарный спрос 95
  поколение   50: лучшая 0.6375, средняя 0.2926
  поколение  100: лучшая 0.6986, средняя 0.3948
  поколение  150: лучшая 0.7492, средняя 0.5787
  поколение  200: лучшая 0.7492, средняя 0.5813
ГА завершён: 247 поколений, 5815 оценок, 1.0 с, причина останова — стагнация 120
поколений
Жадный (ближ. сосед)   пробег    228.1 км | опозданий  10 (  331.5 мин) | перегруз
0 | Q
        = 0.481
ГА (fuzzy)             пробег    222.5 км | опозданий   1 (    2.5 мин) | перегруз
0 | Q
        = 0.749
  Машина 1: склад → 11 → 7 → 14 → 12 → 9 → 20 → 18 → склад
  Машина 2: склад → 6 → 10 → 1 → 15 → 4 → 2 → склад
  Машина 3: склад → 8 → 3 → 17 → 5 → 16 → 13 → 19 → склад

Протокол нечёткого вывода:
  Протяжённость                  =  0.975  →  короткая 0.42, средняя 0.58
  Загрузка транспорта            =  1.000  →  нормальная 1.00
  Соблюдение временных окон      =  0.002  →  хорошее 0.98, удовлетворительное 0.02
  Обслуживание срочных заказов   =  0.164  →  хорошее 0.43, среднее 0.57
  Сбалансированность             =  0.062  →  равномерная 0.97, неравномерная 0.03
  П1  c = 0.415, d = 0.415: ЕСЛИ Протяжённость = «короткая» И Соблюдение временных
окон =
        «хорошее» ТО Качество решения = «очень высокое» (F = 1)
  П2  c = 0.018, d = 0.018: ЕСЛИ Протяжённость = «короткая» И Соблюдение временных
окон =
        «удовлетворительное» ТО Качество решения = «высокое» (F = 1)
  П4  c = 0.585, d = 0.585: ЕСЛИ Протяжённость = «средняя» И Соблюдение временных
окон =
        «хорошее» ТО Качество решения = «высокое» (F = 1)
  П5  c = 0.018, d = 0.018: ЕСЛИ Протяжённость = «средняя» И Соблюдение временных
окон =
        «удовлетворительное» ТО Качество решения = «среднее» (F = 1)
  П10 c = 0.431, d = 0.345: ЕСЛИ Обслуживание срочных заказов = «хорошее» И
Соблюдение
        временных окон = «хорошее» ТО Качество решения = «очень высокое» (F = 0.8)
  П11 c = 0.415, d = 0.291: ЕСЛИ Обслуживание срочных заказов = «хорошее» И
Протяжённость =
        «короткая» ТО Качество решения = «высокое» (F = 0.7)
  П12 c = 0.569, d = 0.342: ЕСЛИ Обслуживание срочных заказов = «среднее» И
Соблюдение
        временных окон = «хорошее» ТО Качество решения = «высокое» (F = 0.6)
  П13 c = 0.018, d = 0.011: ЕСЛИ Обслуживание срочных заказов = «среднее» И
Соблюдение
        временных окон = «удовлетворительное» ТО Качество решения = «среднее» (F =
0.6)
  П17 c = 0.415, d = 0.332: ЕСЛИ Загрузка транспорта = «нормальная» И Протяжённость
=
        «короткая» И Соблюдение временных окон = «хорошее» ТО Качество решения =
«очень
```

```
        высокое» (F = 0.8)
  П19 c = 0.966, d = 0.676: ЕСЛИ Сбалансированность = «равномерная» И Соблюдение
временных
        окон = «хорошее» ТО Качество решения = «высокое» (F = 0.7)
  П20 c = 0.415, d = 0.249: ЕСЛИ Сбалансированность = «равномерная» И Протяжённость
=
        «короткая» ТО Качество решения = «высокое» (F = 0.6)
  П21 c = 0.034, d = 0.014: ЕСЛИ Сбалансированность = «неравномерная» ТО Качество
решения =
        «низкое» (F = 0.4)
  П22 c = 0.018, d = 0.012: ЕСЛИ Сбалансированность = «неравномерная» И Соблюдение
        временных окон = «удовлетворительное» ТО Качество решения = «низкое» (F =
0.7)
  П23 c = 0.034, d = 0.021: ЕСЛИ Сбалансированность = «неравномерная» И
Протяжённость =
        «средняя» ТО Качество решения = «низкое» (F = 0.6)
  П24 c = 0.415, d = 0.374: ЕСЛИ Протяжённость = «короткая» И Обслуживание срочных
заказов
        = «хорошее» И Сбалансированность = «равномерная» ТО Качество решения =
«очень
        высокое» (F = 0.9)
  П26 c = 0.431, d = 0.258: ЕСЛИ Протяжённость = «средняя» И Обслуживание срочных
заказов =
        «хорошее» ТО Качество решения = «высокое» (F = 0.6)
  Дефаззификация (центр тяжести): Q = 0.7492
```